IntegraViz 3D

Plano completo de projeto e execução

Visualizador interativo de integrais duplas e triplas, superfícies, gradientes e equações diferenciais no plano e no espaço (estilo GeoGebra).

O que este documento contém

- Escolha da linguagem e da stack (TypeScript + Three.js) com justificativa
- Fundamentos matemáticos e métodos numéricos a implementar
- Funcionalidades por módulo, com prioridades (MoSCoW)
- Arquitetura, estrutura de pastas, tipos e trechos de código
- Configuração completa do VSCode, testes, CI e deploy
- Roadmap de 10 fases com tarefas e critérios de aceite
- Casos de teste com valores exatos, riscos, backlog e referências

Versão 1.0 · 15 de outubro de 2026 · Nome de trabalho do projeto: IntegraViz 3D

Sumário

1. Visão geral do projeto 5

1.1 A ideia em uma frase 5

1.2 Problema que o projeto resolve 5

1.3 Público-alvo 5

1.4 Objetivos 5

1.5 O que está fora do escopo da versão 1 6

1.6 Critério de sucesso do MVP 6

2. Escolha da linguagem e da stack 7

2.1 Opções avaliadas 7

2.2 Decisão 7

2.3 Stack completa 7

3. Fundamentos matemáticos que o software implementa 9

3.1 Integral dupla 9

3.2 Integral tripla 9

3.3 Cálculo diferencial multivariável 10

3.4 Equações diferenciais ordinárias 10

3.5 Métodos numéricos: quando usar cada um 10

4. Funcionalidades por módulo 12

4.1 Módulo A — Funções e superfícies 12

4.2 Módulo B — Integral dupla 12

4.3 Módulo C — Integral tripla 13

4.4 Módulo D — Cálculo diferencial multivariável 13

4.5 Módulo E — Equações diferenciais 13

4.6 Módulo F — Plataforma 14

5. Arquitetura de software 15

5.1 Princípios 15

5.2 Camadas e fluxo de dados 15

5.3 Módulos principais e responsabilidades 16

5.4 Regra de dependências (imposta pelo ESLint) 16

6. Estrutura de pastas do repositório 17

7. Ambiente de desenvolvimento no VSCode 18

7.1 Pré-requisitos 18

7.2 Criando o projeto 18

7.3 Extensões recomendadas do VSCode 18

7.4 Arquivos de configuração do workspace 19

7.5 Fluxo de trabalho com Git e CI . . . 20
8. Modelo de dados (tipos centrais) . . . 21
9. Motor de expressões e derivadas . . . 23
9.1 Do texto à função rápida . . . 23
9.2 Derivadas e gradiente: três camadas . . . 24
9.3 Regiões e pertinência . . . 24
10. Motor numérico em detalhe . . . 25
10.1 Quadratura de Gauss–Legendre . . . 25
10.2 Integral dupla iterada (região tipo I) . . . 25
10.3 Soma de Riemann para visualização . . . 26
10.4 Integral tripla . . . 26
10.5 EDOs . . . 27
11. Renderização 3D com Three.js . . . 28
11.1 Cena, câmera e convenções . . . 28
11.2 Superfície z = f(x, y) . . . 28
11.3 Prismas de Riemann com InstancedMesh . . . 28
11.4 Sólidos, fatias e nuvens de pontos . . . 29
11.5 Vetores, setas e trilhas . . . 29
11.6 Desempenho de renderização . . . 30
12. Projeto da interface (UX) . . . 31
12.1 Layout . . . 31
12.2 Painel de expressões (coração da interface) . . . 31
12.3 Fluxos de uso principais . . . 31
12.4 Acessibilidade, internacionalização e responsividade . . . 32
13. Roadmap de execução por fases . . . 33
Fase 0 — Preparação (1 semana) . . . 33
Fase 1 — Fundação e superfícies (2 semanas) . . . 34
Fase 2 — Integral dupla retangular (3 semanas) . . . 34
Fase 3 — Regiões gerais e polar (3 semanas) . . . 35
Fase 4 — Cálculo diferencial multivariável (2 semanas) . . . 35
Fase 5 — Integral tripla (4 semanas) . . . 35
Fase 6 — Equações diferenciais (4 semanas) . . . 36
Fase 7 — Plataforma e polimento (3 semanas) . . . 36
Fase 8 — Desempenho e WebAssembly (3 semanas) . . . 36
Fase 9 — Extras contínuos . . . 36
Definição de "pronto" para cada funcionalidade . . . 36
14. Estratégia de testes e validação . . . 37

14.1 Pirâmide de testes . . . 37
14.2 Casos de referência com valor exato . . . 37
14.3 Testes de propriedades (fast-check) . . . 38
14.4 Ordem de convergência . . . 38
14.5 Testes visuais . . . 39
14.6 Validação cruzada com Python . . . 39
15. Desempenho: orçamentos e técnicas . . . 40
15.1 Técnicas de otimização de laços numéricos . . . 40
15.2 Experiência durante a interação . . . 40
16. Publicação e distribuição . . . 41
17. Riscos e mitigações . . . 42
18. Backlog de ideias futuras . . . 43
19. Checklist da primeira semana (dia a dia) . . . 44
20. Referências e fontes de estudo . . . 45
Livros . . . 45
Artigos clássicos . . . 45
Documentação e tutoriais . . . 45
Ferramentas para se inspirar (e se diferenciar) . . . 45
Apêndice A — Folha rápida de fórmulas e convenções . . . 46
Apêndice B — Biblioteca de exemplos para o app . . . 47
B.1 Superfícies e integrais . . . 47
B.2 Equações diferenciais . . . 47
Apêndice C — Glossário . . . 49

1. Visão geral do projeto

1.1 A ideia em uma frase

Construir um **"GeoGebra do Cálculo Multivariável e das Equações Diferenciais"**: uma aplicação interativa em que a pessoa digita uma função, define uma região, e **vê** em 3D a superfície, os prismas de Riemann que aproximam a integral dupla, o sólido de uma integral tripla, o campo gradiente, o plano tangente, as curvas de nível, os campos de direções e as trajetórias de uma EDO, tudo atualizando em tempo real enquanto sliders e parâmetros são movidos.

O nome de trabalho usado neste documento é **IntegraViz 3D**. Pode ser trocado a qualquer momento; ele só aparece no código como o nome do pacote e no título da aplicação.

1.2 Problema que o projeto resolve

- Integrais múltiplas são difíceis de *enxergar*: o aluno aprende a montar os limites (tipo I, tipo II, polar, esféricas) sem nunca ver a região nem o sólido que está integrando.
- O GeoGebra 3D e o Desmos 3D desenham superfícies muito bem, mas **não foram feitos** para mostrar somas de Riemann 3D, mudança de ordem de integração, Jacobianos ou convergência numérica.
- Ferramentas de EDO (como o pplane) são antigas, 2D e desconectadas do restante do cálculo.
- Falta uma ferramenta única que una: superfície, gradiente, integral dupla, integral tripla e EDOs no mesmo ambiente, com a mesma sintaxe de entrada.

1.3 Público-alvo

- Estudantes de Cálculo II/III, Cálculo Vetorial, Equações Diferenciais e Física (engenharias, exatas).
- Professores que querem mostrar em sala, projetando a tela, o que acontece quando n cresce ou a região muda.
- O próprio autor do projeto, que vai aprender muito de matemática numérica e computação gráfica.

1.4 Objetivos

#	Objetivo	Como medimos
O1	Visualizar z = f(x, y) e superfícies paramétricas/implícitas em 3D com interação fluida.	60 fps em notebook comum com malha 200×200; atualização < 100 ms ao editar a fórmula.
O2	Integral dupla com prismas de Riemann, regiões gerais (retangular, tipo I, II, polar) e valor numérico confiável.	Erro relativo < 1e-8 nos casos de teste com valor exato conhecido.
O3	Integral tripla em coordenadas cartesianas, cilíndricas e esféricas, com visualização do sólido e fatias.	Volume da esfera unitária = 4π/3 com erro < 1e-6.
O4	Gradiente, derivada direcional, plano tangente, curvas de nível, Hessiana e pontos críticos.	Gradiente simbólico confere com diferenças finitas em 100% dos testes.
O5	EDOs: campo de direções, soluções por clique, sistemas 2D e 3D, retrato de fase, equilíbrios.	RK4 apresenta ordem de convergência ≈ 4 no teste y' = y.

#	Objetivo	Como medimos
O6	Rodar 100% no navegador, sem instalação, com link compartilhável do estado.	Abrir um link e ver exatamente a mesma cena.

1.5 O que está fora do escopo da versão 1

- Sistema de álgebra computacional (CAS) completo. A v1 entrega **valor numérico** e, quando possível, o valor exato por um módulo opcional (Fase 9).
- Integrais de linha e de superfície, teoremas de Green/Stokes/Gauss (ficam no backlog, Seção 18).
- EDPs (calor, onda) — previstas como extensão natural, depois das EDOs.
- Aplicativo móvel nativo (a versão web já funciona em telas touch).
- Contas de usuário e nuvem. O estado é salvo em arquivo JSON e no próprio link.

1.6 Critério de sucesso do MVP

MVP (fim da Fase 3 — cerca de 8 a 10 semanas)
Digitar uma f(x, y), escolher uma região (retângulo, tipo I, tipo II ou polar), ver a superfície e os prismas de Riemann, mover o slider de n, ler o valor aproximado, o erro estimado e um gráfico de convergência. Tudo em português, no navegador, com testes automatizados cobrindo os valores exatos.

2. Escolha da linguagem e da stack

2.1 Opções avaliadas

A escolha depende de três exigências: (1) **interatividade em tempo real** estilo GeoGebra; (2) **renderização 3D** de qualidade; (3) **facilidade de compartilhar** (abrir um link e usar). A tabela compara as alternativas realistas para um projeto feito inteiramente no VSCode.

Opção	Prós	Contras	Veredito
TypeScript + Three.js (web)	Roda em qualquer navegador; ecossistema 3D maduro; UI rica com React; deploy trivial; VSCode é o melhor editor para TS; Web Workers dão paralelismo; WASM acelera os trechos críticos.	Matemática numérica em JS é menos rápida que C++/Rust puro; sem CAS nativo.	**Escolhida**
Python (Plotly, PyVista, SymPy)	SymPy dá integrais simbólicas exatas; NumPy/SciPy são excelentes.	Interação tipo GeoGebra é limitada; distribuir app desktop é trabalhoso; GUI pesada.	Ótima para protótipo/validação, não para o produto final.
Julia (Makie)	Muito rápida e feita para cálculo científico.	Ecossistema de UI e distribuição web é pequeno; empacotar é difícil.	Descartada
C++ (Qt + OpenGL/VTK)	Desempenho máximo.	Ciclo de desenvolvimento lento, build complexo, sem compartilhamento por link.	Descartada
Rust (wgpu/egui/Bevy)	Rápida, segura, compila para WASM e nativo.	Curva de aprendizado alta; poucas bibliotecas de parsing matemático e UI de fórmulas.	Usar só em módulos de cálculo pesado (Fase 8).
C# / Unity	Bom motor 3D.	Pesado para um app de matemática; web export grande.	Descartada

2.2 Decisão

Linguagem principal: TypeScript (modo strict)
Interface em React + Vite, renderização com Three.js, cálculo pesado em Web Workers. Opcional e tardio: Rust compilado para WebAssembly para integrais triplas, Monte Carlo e marching cubes; e SymPy via Pyodide para valores exatos. O núcleo matemático é escrito como biblioteca pura (sem DOM, sem Three.js), o que permite reaproveitá-lo em Node, em testes ou em outra interface.

2.3 Stack completa

Camada	Tecnologia	Motivo
Linguagem	TypeScript (strict)	Tipagem forte evita bugs de índices, domínios e unidades.
Build / dev server	Vite	Hot reload instantâneo; suporte nativo a Workers e WASM.
UI	React + Tailwind CSS	Painéis, listas de expressões e formulários com pouco código.

Camada	Tecnologia	Motivo
Estado	Zustand (+ Immer opcional)	Simples, sem boilerplate, fácil de serializar para JSON/URL.
3D	Three.js (uso direto, sem react-three-fiber)	Controle total sobre buffers, instancing e shaders; menos camadas entre você e a GPU.
Parsing de fórmulas	math.js (parse + derivative)	Gera AST, deriva simbolicamente e aceita sintaxe natural (2x, x^2, sin(x)).
Edição de fórmulas	MathLive (campo de equação visual) + KaTeX (renderização)	Digitar frações, raízes e integrais como no caderno; gera LaTeX e texto.
Paralelismo	Web Workers + Comlink	Chamadas assíncronas tipadas; UI nunca trava.
Gráficos 2D auxiliares	uPlot ou Canvas próprio	Convergência da soma de Riemann, séries temporais de EDO.
Testes	Vitest, fast-check (propriedades), Playwright (E2E)	Cobre matemática, UI e regressões visuais.
Qualidade	ESLint, Prettier, TypeScript strict, Husky	Padroniza código e bloqueia commits quebrados.
Deploy	Vercel, Netlify ou GitHub Pages	Gratuito para projetos estáticos; PWA opcional.
Opcionais	Rust + wasm-bindgen; Pyodide + SymPy	Desempenho (Fase 8) e valores exatos (Fase 9).

3. Fundamentos matemáticos que o software implementa

Esta seção é o "contrato matemático" do projeto: tudo que a aplicação calcula ou desenha deve poder ser rastreado até uma fórmula daqui. Ela também serve de guia de estudo para quem vai programar.

3.1 Integral dupla

- **Definição (soma de Riemann):** $\iint_R f(x, y)\, dA = \lim \Sigma_i \Sigma_j f(x^*_{ij}, y^*_{ij})\, \Delta A$. O ponto de amostra (x*, y*) pode ser canto inferior-esquerdo, canto superior-direito, centro da célula, mínimo ou máximo da célula (somas inferior e superior) ou aleatório.
- **Fubini** (f contínua em R = [a, b] × [c, d]): $\iint f\, dA = \int_a^b \int_c^d f\, dy\, dx = \int_c^d \int_a^b f\, dx\, dy$.
- **Região tipo I:** $D = \{a \le x \le b,\ g_1(x) \le y \le g_2(x)\} \rightarrow \int_a^b \int_{g1(x)}^{g2(x)} f\, dy\, dx$.
- **Região tipo II:** $D = \{c \le y \le d,\ h_1(y) \le x \le h_2(y)\} \rightarrow \int_c^d \int_{h1(y)}^{h2(y)} f\, dx\, dy$.
- **Polar:** $x = r \cos\theta$, $y = r \operatorname{sen}\theta$, $dA = \mathbf{r}\, dr\, d\theta$.
- **Mudança de variáveis geral:** $\iint f(x, y)\, dA = \iint f(x(u, v), y(u, v)) \cdot |\partial(x, y)/\partial(u, v)|\, du\, dv$.

Aplicações que viram recursos do app:

- Volume sob a superfície (f ≥ 0) e área da região (f = 1).
- Massa com densidade ρ(x, y): $m = \iint \rho\, dA$. Centro de massa: $\bar{x} = (1/m)\iint x\rho\, dA$, $\bar{y} = (1/m)\iint y\rho\, dA$.
- Momentos de inércia: $I_x = \iint y^2\rho\, dA$, $I_y = \iint x^2\rho\, dA$, $I_0 = \iint (x^2 + y^2)\rho\, dA$.
- Valor médio: $f_{méd} = (1/A)\iint f\, dA$.
- Área de superfície: $A_S = \iint \sqrt{(1 + f_x^{\,2} + f_y^{\,2})}\, dA$.

3.2 Integral tripla

- **Definição:** soma de Riemann com caixas $\Delta V = \Delta x\, \Delta y\, \Delta z$. O Teorema de Fubini permite 6 ordens de integração (dx dy dz, dx dz dy, ...).
- **Região tipo 1:** $E = \{(x, y) \in D,\ u_1(x, y) \le z \le u_2(x, y)\}$; tipos 2 e 3 são análogos trocando os papéis das variáveis.
- **Cilíndricas:** $x = r \cos\theta$, $y = r \operatorname{sen}\theta$, $z = z$; $dV = \mathbf{r}\, dz\, dr\, d\theta$.
- **Esféricas** (convenção física/ISO: ρ = distância à origem, φ = ângulo a partir do eixo +z em [0, π], θ = azimute em [0, 2π]): $x = \rho \operatorname{sen}\varphi \cos\theta$, $y = \rho \operatorname{sen}\varphi \operatorname{sen}\theta$, $z = \rho \cos\varphi$; $dV = \boldsymbol{\rho^2 \operatorname{sen}\varphi}\, d\rho\, d\varphi\, d\theta$.
- **Aplicações:** volume, massa $m = \iiint \rho\, dV$, centro de massa, momento de inércia $I_z = \iiint (x^2 + y^2)\rho\, dV$, valor médio de uma função no sólido.

> **Atenção: convenções de ângulos**
> Livros de Cálculo (Stewart, Thomas) usam φ como ângulo polar e θ como azimute, como acima. Livros de outras áreas invertem os símbolos. O app deve mostrar, ao lado de qualquer integral esférica, um pequeno diagrama com a convenção usada e permitir alternar para a convenção de matemática pura.

3.3 Cálculo diferencial multivariável

- **Gradiente:** $\nabla f = (f_x, f_y[, f_z])$. Aponta na direção de maior crescimento e é perpendicular às curvas (superfícies) de nível; $|\nabla f|$ é a taxa máxima de variação.
- **Derivada direcional:** $D_u f = \nabla f \cdot u$, com $|u| = 1$.
- **Plano tangente / linearização:** $z = f(a, b) + f_x(a, b)(x - a) + f_y(a, b)(y - b)$.
- **Hessiana e classificação de pontos críticos** ($\nabla f = 0$): $D = f_{xx} f_{yy} - f_{xy}^2$. $D > 0$ e $f_{xx} > 0$ → mínimo local; $D > 0$ e $f_{xx} < 0$ → máximo local; $D < 0$ → sela; $D = 0$ → inconclusivo.
- **Multiplicadores de Lagrange:** $\nabla f = \lambda \nabla g$ com $g = c$ (visualizar a curva de nível de f tangente à restrição).
- **Descida do gradiente:** $p_{k+1} = p_k - \alpha \nabla f(p_k)$ (animação de uma bola "rolando" pela superfície).
- **Campos vetoriais:** divergente $\nabla \cdot F$ e rotacional $\nabla \times F$, com linhas de corrente.

3.4 Equações diferenciais ordinárias

- **1ª ordem:** $y' = f(t, y)$. Campo de direções (um segmento de reta por ponto da malha), isóclinas e solução do PVI $y(t_0) = y_0$. Mostrar também casos em que a unicidade falha (ex.: $y' = \sqrt{|y|}$ em $y = 0$).
- **Sistemas autônomos 2D:** $x' = P(x, y)$, $y' = Q(x, y)$. Retrato de fase, nuclinas, pontos de equilíbrio ($P = Q = 0$), matriz Jacobiana J e seus autovalores → nó, sela, foco/espiral, centro; estável ou instável.
- **Sistemas 3D:** Lorenz ($x' = \sigma(y - x)$, $y' = x(\rho - z) - y$, $z' = xy - \beta z$), Rössler, sistemas lineares 3×3.
- **2ª ordem:** $y'' = F(t, y, y')$ vira o sistema (y, v) com $v = y'$. Exemplos: oscilador harmônico, pêndulo, Van der Pol, Duffing.
- **Métodos numéricos:** Euler (ordem 1), Heun/RK2 (ordem 2), RK4 (ordem 4), Dormand–Prince RK45 (passo adaptativo). O erro global de um método de ordem p é $O(h^p)$.
- **EDPs (extensão futura):** calor $u_t = \alpha u_{xx}$ e onda $u_{tt} = c^2 u_{xx}$ aparecem naturalmente como superfícies $z = u(x, t)$ animadas. Diferenças finitas explícitas exigem $\alpha \Delta t / \Delta x^2 \leq 1/2$ para estabilidade.

3.5 Métodos numéricos: quando usar cada um

Problema	Método	Ordem / erro	Quando usar
Integral 1D suave	Gauss–Legendre de n pontos	Exato para polinômios de grau ≤ 2n − 1	Padrão para integrandos suaves
Integral 1D com singularidade branda	Gauss–Kronrod G7K15 adaptativo	Estimativa de erro embutida	Fallback automático
Integral dupla em região tipo I/II	Gauss–Legendre aninhado	Exponencial se suave	Caso mais comum
Integral dupla/tripla em região complicada	Monte Carlo / quase-Monte Carlo (Halton, Sobol)	$O(1/\sqrt{N})$ / ≈ $O(1/N)$	Regiões dadas por desigualdades
Integral tripla suave	Produto tensorial G-L ou cubatura adaptativa (Genz–Malik)	Depende da suavidade	Cartesiano, cilíndrico, esférico
Soma de Riemann (visual)	Ponto médio / esquerda / direita	$O(h^2)$ (médio), $O(h)$ (outros)	Apenas para ilustrar

Problema	Método	Ordem / erro	Quando usar
EDO	RK4 passo fixo; RK45 adaptativo	$O(h^4)$; controle de tolerância	Padrão; Euler/Heun p/ comparação
EDO rígida	Implícito (Rosenbrock, BDF)	Estável com h grande	Opcional (Fase 9)
Raízes / equilíbrios	Newton com sementes em malha	Quadrática	Pontos críticos e equilíbrios
Superfície implícita F = 0	Marching cubes / squares	Linear no tamanho da malha	Sólidos e curvas de nível

4. Funcionalidades por módulo

As funcionalidades estão agrupadas em módulos independentes. A prioridade segue o método MoSCoW: **M** (precisa ter), **S** (deveria ter), **C** (pode ter), **W** (não nesta versão).

4.1 Módulo A — Funções e superfícies

Recurso	Descrição	Prio.
Superfície explícita	z = f(x, y) com domínio retangular editável e resolução ajustável.	M
Parâmetros livres	Qualquer símbolo que não seja x, y, z ou constante vira slider automático (a, b, c...).	M
Mapa de cores	Por altura, por |∇f| ou por curvatura; paletas viridis, magma, coolwarm, arco-íris.	M
Eixos, grade e rótulos	Eixos X, Y, Z com ticks "bonitos", planos de grade, câmera com Z para cima.	M
Curvas de nível	Projetadas no plano z = 0 e/ou elevadas sobre a superfície; passo configurável.	S
Cortes	Plano x = c, y = c ou z = c com slider; mostra a curva de interseção.	S
Superfície paramétrica	r(u, v) = (x(u, v), y(u, v), z(u, v)): esfera, toro, faixa de Möbius.	S
Superfície implícita	F(x, y, z) = 0 via marching cubes.	S
Transparência e wireframe	Opacidade, malha de arame e linhas de nível no shader.	S
Superfícies múltiplas	Várias funções simultâneas (ex.: região entre dois paraboloides).	M

4.2 Módulo B — Integral dupla

Recurso	Descrição	Prio.
Região retangular	R = [a, b] × [c, d] com campos numéricos e arrasto no plano.	M
Prismas de Riemann	Colunas sob a superfície com n × m divisões; modos: esquerda, direita, centro, inferior, superior, aleatório.	M
Valor e erro	Soma de Riemann, valor de alta precisão (Gauss–Legendre), erro absoluto e relativo.	M
Gráfico de convergência	Soma × n; erro × n em escala log-log, com a inclinação (ordem) estimada.	S
Região tipo I / II	Limites dados por funções g1(x), g2(x) ou h1(y), h2(y); a região é desenhada no plano.	M
Região polar	r de r1(θ) a r2(θ); mostra o Jacobiano r como "setores" de área variável.	M
Região por desigualdades	Ex.: $x^2 + y^2 \leq 4$ e $y \geq x$; usa varredura + bissecção ou Monte Carlo.	S
Mudar a ordem de integração	Transforma dy dx em dx dy e mostra as duas regiões/limites lado a lado.	S
Sólido preenchido	Mostra o volume exato como sólido translúcido entre z = 0 e a superfície.	S
Aplicações físicas	Massa, centro de massa, momentos de inércia, valor médio, área de superfície.	S

Recurso	Descrição	Prio.
Passo a passo	Exibe a integral iterada montada, com os limites substituídos (texto + LaTeX).	C

4.3 Módulo C — Integral tripla

Recurso	Descrição	Prio.
Regiões sólidas tipo 1/2/3	Limites u1(x, y) ≤ z ≤ u2(x, y) sobre uma região D do plano.	M
Coordenadas	Cartesianas, cilíndricas e esféricas, com Jacobiano mostrado.	M
Visualização do sólido	Superfícies de contorno translúcidas; fatias paralelas a um plano.	M
Nuvem de pontos	Amostras Monte Carlo coloridas pelo valor do integrando.	S
Corte por plano	Slider para "abrir" o sólido e ver o interior.	S
Volumes de referência	Presets: esfera, cone, paraboloide, toro, interseção de cilindros.	S
Densidade variável	Massa e centro de massa com ρ(x, y, z), cor por densidade.	S
Região implícita	Desigualdades como $x^2 + y^2 + z^2 \leq 1$ e $z \geq \sqrt{(x^2 + y^2)}$.	S
Animação de fatias	Varre z de z_min a z_max mostrando a integral dupla de cada fatia (Princípio de Cavalieri).	C

4.4 Módulo D — Cálculo diferencial multivariável

- **(M)** Ponto móvel sobre a superfície (arrastar ou clicar) com leitura de f, f_x, f_y.
- **(M)** Vetor gradiente desenhado no ponto e **campo de gradientes** como setas sobre o plano xy.
- **(M)** Derivada direcional: seletor de ângulo (círculo unitário) e reta tangente naquela direção.
- **(M)** Plano tangente translúcido com slider de "zoom" para mostrar a aproximação linear.
- **(S)** Hessiana, classificação automática de pontos críticos com marcadores (mínimo, máximo, sela).
- **(S)** Descida/subida do gradiente animada, com passo α ajustável.
- **(C)** Multiplicadores de Lagrange: restrição g(x, y) = c e curva de nível de f tangente.
- **(C)** Divergente e rotacional de campos vetoriais 2D/3D; linhas de corrente.

4.5 Módulo E — Equações diferenciais

Recurso	Descrição	Prio.
Campo de direções 1ª ordem	y' = f(t, y); clicar no plano cria uma solução com aquela condição inicial.	M
Métodos selecionáveis	Euler, Heun, RK4, RK45; comparar até 3 métodos e a solução exata (se informada).	M
Sistemas 2D e retrato de fase	Trajetórias, nuclinas, equilíbrios automáticos, Jacobiana e classificação.	M
Sistemas 3D	Lorenz, Rössler etc. com trilha 3D animada e rotação livre da câmera.	M
EDO de 2ª ordem	Converte automaticamente para sistema; gráficos y(t), y'(t) e plano de fase.	M

Recurso	Descrição	Prio.
Parâmetros	Sliders para σ, ρ, β, amortecimento etc.; a trajetória atualiza em tempo real.	S
Solução exata opcional	O usuário informa y(t); o app plota o erro numérico.	S
Bifurcações simples	Varrer um parâmetro e plotar equilíbrios (diagrama de bifurcação 1D).	C
Superfície de solução (EDP)	u(x, t) do calor/onda como superfície animada.	C

4.6 Módulo F — Plataforma

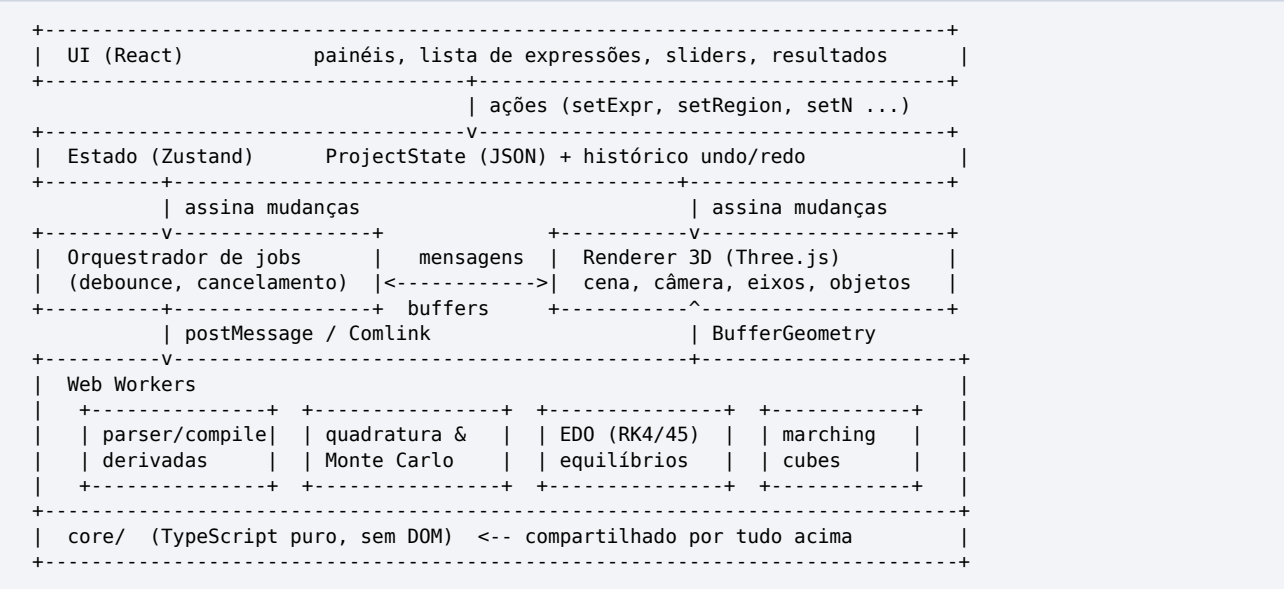
- **(M)** Salvar/abrir projeto em JSON; estado codificado no link (hash da URL, comprimido).
- **(M)** Desfazer/refazer, atalhos de teclado, modo claro/escuro.
- **(S)** Exportar imagem PNG, vídeo WebM (MediaRecorder) e modelo 3D (GLTF).
- **(S)** Biblioteca de exemplos prontos (Apêndice B) e tutorial interativo inicial.
- **(S)** Internacionalização (pt-BR e en) e acessibilidade (teclado, contraste, redução de movimento).
- **(C)** Modo apresentação (fonte grande, painéis ocultos) e modo exercício (o app confere a resposta do aluno).
- **(C)** PWA para uso offline.

5. Arquitetura de software

5.1 Princípios

1. **Núcleo matemático puro:** nada de DOM, React ou Three.js dentro de `core/`. Isso o torna testável, portável e rápido.
2. **Dados imutáveis e serializáveis:** o estado do projeto é um objeto JSON simples; é o que vai para arquivo, histórico de desfazer e link.
3. **Cálculo fora da thread principal:** qualquer tarefa que possa levar mais de ~8 ms vai para um Worker, com cancelamento (quando o usuário mexe de novo, o trabalho antigo é descartado).
4. **Typed arrays e transferíveis:** resultados de malhas e trajetórias trafegam como Float32Array/Float64Array sem cópia.
5. **Renderizar só quando necessário:** sem animação ativa, a cena não redesenha a 60 fps.
6. **Cada módulo matemático tem seu "construtor de geometria":** função pura que transforma resultados numéricos em buffers prontos para a GPU.

5.2 Camadas e fluxo de dados



Fluxo típico (editar uma fórmula): o usuário digita → o campo valida (parse) e mostra erro em português se houver → o estado é atualizado → o orquestrador aguarda 120 ms de inatividade (debounce) → envia ao Worker um job {id, tipo, expressão, parâmetros, domínio, resolução} → o Worker compila a expressão, avalia a malha, calcula normais e valores → devolve Float32Arrays como transferíveis → o renderer atualiza a BufferGeometry → o painel de resultados mostra tempo e valores. Se o usuário digitar de novo antes do término, o job antigo é cancelado (o id mais novo vence).

5.3 Módulos principais e responsabilidades

Módulo	Responsabilidade	Depende de
core/expr	Parse, validação, geração de código JS seguro, derivadas simbólicas, diferenciação automática.	math.js
core/regions	Definição de regiões 2D/3D (tipo I/II/polar/implícita), teste de pertinência, limites por varredura.	core/expr
core/quad	Gauss–Legendre, Gauss–Kronrod, integrais iteradas, Jacobianos, Monte Carlo/QMC.	core/regions
core/diff	Gradiente, Hessiana, plano tangente, pontos críticos, descida do gradiente.	core/expr
core/ode	Euler, Heun, RK4, RK45, eventos, equilíbrios, Jacobiana e autovalores.	core/expr
core/mesh	Construtores de malha: superfície, prismas, sólido, marching cubes/squares, setas, nível.	core/*
render/	Cena, câmera, controles, eixos, materiais, shaders, picking, exportação.	Three.js
workers/	Pontos de entrada dos Workers e protocolo de mensagens tipado.	core/*
state/	Store Zustand, serialização, histórico, migração de versões do arquivo.	—
ui/	Componentes React: painéis, campos de fórmula, toolbars, gráficos 2D.	state/, render/
i18n/	Textos pt-BR e en; mensagens de erro matemático.	—

5.4 Regra de dependências (imposta pelo ESLint)

```
ui      --> state, render, i18n
render  --> core (somente tipos), three
workers --> core
state   --> core (somente tipos)
core    --> (nada fora de math.js e bibliotecas puras)   // NUNCA importa ui/render/state
```

6. Estrutura de pastas do repositório

```
integraviz/
├─ .github/workflows/ci.yml          # lint + testes + build a cada push
├─ .vscode/
│   ├─ settings.json                 # formatação, ESLint, paths
│   ├─ launch.json                   # debug no Chrome
│   ├─ tasks.json                    # atalhos: dev, test, build
│   └─ extensions.json               # extensões recomendadas
├─ public/                           # ícones, exemplos .json
├─ docs/                             # decisões (ADR), este plano, notas de matemática
├─ src/
│   ├─ core/
│   │   ├─ expr/                     # parse.ts, codegen.ts, derivative.ts, autodiff.ts
│   │   ├─ regions/                  # region2d.ts, region3d.ts, implicit.ts
│   │   ├─ quad/                     # gauss.ts, kronrod.ts, iterated.ts, montecarlo.ts, jacobians.ts
│   │   ├─ diff/                     # gradient.ts, hessian.ts, critical.ts, descent.ts
│   │   ├─ ode/                      # euler.ts, rk4.ts, rk45.ts, equilibria.ts, linearize.ts
│   │   ├─ mesh/                     # surface.ts, riemann.ts, solid.ts, marching.ts, arrows.ts, contour.ts
│   │   └─ types.ts
│   ├─ workers/
│   │   ├─ protocol.ts               # tipos de Job/Result
│   │   ├─ compute.worker.ts
│   │   └─ client.ts                 # orquestrador (debounce, cancelamento)
│   ├─ render/
│   │   ├─ scene.ts, camera.ts, axes.ts, labels.ts
│   │   ├─ objects/                  # surfaceObject.ts, riemannObject.ts, vectorField.ts, trail.ts
│   │   ├─ shaders/                  # surface.vert.glsl, surface.frag.glsl, colormaps.ts
│   │   ├─ picking.ts, export.ts
│   ├─ state/                        # store.ts, schema.ts, serialize.ts, history.ts
│   ├─ ui/
│   │   ├─ App.tsx, Toolbar.tsx, ExprList.tsx, MathField.tsx
│   │   ├─ panels/                   # SurfacePanel, DoubleIntegralPanel, TripleIntegralPanel, OdePanel
│   │   ├─ charts/                   # ConvergenceChart, TimeSeries
│   │   └─ examples/                 # biblioteca de exemplos
│   ├─ i18n/                         # pt-BR.json, en.json
│   └─ main.tsx
├─ tests/
│   ├─ unit/                         # matemática: valores exatos, ordem de convergência
│   ├─ property/                     # fast-check: Fubini, linearidade, polar = cartesiano
│   ├─ e2e/                          # Playwright
│   └─ fixtures/                     # casos de referência (JSON)
├─ index.html, vite.config.ts, tsconfig.json, eslint.config.js, package.json
└─ README.md
```

7. Ambiente de desenvolvimento no VSCode

7.1 Pré-requisitos

- **Node.js (versão LTS atual)** e o gerenciador **pnpm** (ou npm, se preferir menos configuração).
- **Git** e uma conta no GitHub (repositório, CI e deploy).
- **VSCode** atualizado e um navegador baseado em Chromium (DevTools e depuração do WebGL).
- Opcional, para a Fase 8: **Rust** (rustup) e **wasm-pack**.
- Opcional, para validação cruzada: Python 3 com SymPy e SciPy, usados só como "gabarito" nos testes.

7.2 Criando o projeto

```
# 1) criar o app com Vite + React + TypeScript
pnpm create vite integraviz --template react-ts
cd integraviz && pnpm install

# 2) dependências de execução
pnpm add three mathjs zustand comlink katex mathlive i18next uplot

# 3) dependências de desenvolvimento
pnpm add -D @types/three vitest @vitest/ui jsdom fast-check \
            eslint prettier eslint-plugin-boundaries husky lint-staged \
            @playwright/test tailwindcss

# 4) rodar
pnpm dev            # servidor de desenvolvimento (porta 5173)
pnpm vitest         # testes em modo watch
pnpm build          # build de produção
```

O Tailwind deve ser configurado seguindo o guia oficial da versão instalada (o processo de setup mudou entre versões). Depois de criado, inicialize o Git, faça o primeiro commit e crie o repositório remoto.

7.3 Extensões recomendadas do VSCode

Extensão	Para que serve
ESLint e Prettier	Lint e formatação automática ao salvar.
Error Lens	Mostra erros de TypeScript na própria linha.
Pretty TypeScript Errors	Mensagens de erro de tipos legíveis.
Vitest (Explorer)	Rodar/depurar testes direto na barra lateral, com ícones de ✓ e ✗.
Playwright Test for VSCode	Rodar e gravar testes E2E.
Tailwind CSS IntelliSense	Autocomplete das classes utilitárias.
GLSL / Shader languages support	Realce e checagem de shaders .glsl (superfície, contornos).
GitLens	Histórico e autoria por linha; útil para revisar a própria evolução.
Markdown All in One + Markdown Preview Mermaid	Escrever docs e diagramas em docs/.
LaTeX Workshop (opcional)	Se quiser escrever as notas de matemática em LaTeX.

Extensão	Para que serve
Code Spell Checker (+ dicionário pt-BR)	Evita erros de digitação em textos e mensagens da UI.
Todo Tree	Lista os TODO/FIXME espalhados pelo código.

7.4 Arquivos de configuração do workspace

.vscode/settings.json

```
{
  "editor.formatOnSave": true,
  "editor.defaultFormatter": "esbenp.prettier-vscode",
  "editor.codeActionsOnSave": { "source.fixAll.eslint": "explicit" },
  "typescript.tsdk": "node_modules/typescript/lib",
  "typescript.preferences.importModuleSpecifier": "non-relative",
  "files.eol": "\n",
  "search.exclude": { "**/node_modules": true, "**/dist": true },
  "cSpell.language": "en,pt,pt_BR"
}
```

.vscode/launch.json (depurar no Chrome com o dev server ligado)

```
{
  "version": "0.2.0",
  "configurations": [
    {
      "type": "chrome",
      "request": "launch",
      "name": "IntegraViz no Chrome",
      "url": "http://localhost:5173",
      "webRoot": "${workspaceFolder}/src"
    }
  ]
}
```

.vscode/tasks.json

```
{
  "version": "2.0.0",
  "tasks": [
    { "label": "dev",   "type": "shell", "command": "pnpm dev",        "isBackground": true },
    { "label": "test",  "type": "shell", "command": "pnpm vitest run", "group": "test" },
    { "label": "build", "type": "shell", "command": "pnpm build",      "group": "build" }
  ]
}
```

tsconfig.json (trechos essenciais)

```
{
  "compilerOptions": {
    "target": "ES2022", "module": "ESNext", "moduleResolution": "bundler",
    "strict": true, "noUncheckedIndexedAccess": true, "noImplicitOverride": true,
    "exactOptionalPropertyTypes": true, "noFallthroughCasesInSwitch": true,
    "lib": ["ES2022", "DOM", "DOM.Iterable", "WebWorker"],
    "baseUrl": ".", "paths": { "@/*": ["src/*"] }
  }
}
```

> **Dica: noUncheckedIndexedAccess**
> Com essa opção, `arr[i]` passa a ter tipo `number | undefined`. Em laços numéricos isso incomoda, mas evita uma classe inteira de erros de borda. Em trechos quentes, use o operador `!` de forma consciente ou iteração por índice sobre Float64Array.

7.5 Fluxo de trabalho com Git e CI

- Branches curtas por funcionalidade (`feat/riemann-columns`) e Pull Requests, mesmo trabalhando sozinho: o PR é o seu "checklist de revisão".
- Mensagens no padrão Conventional Commits (`feat:`, `fix:`, `test:`, `perf:`, `docs:`).
- Husky + lint-staged rodam ESLint/Prettier nos arquivos alterados antes de cada commit.
- Cada fase do roadmap termina com uma tag (`v0.1.0`, `v0.2.0`...) e um deploy de preview.

```
# .github/workflows/ci.yml
name: CI
on: [push, pull_request]
jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: pnpm/action-setup@v4
      - uses: actions/setup-node@v4
        with: { node-version: lts/*, cache: pnpm }
      - run: pnpm install --frozen-lockfile
      - run: pnpm lint
      - run: pnpm tsc --noEmit
      - run: pnpm vitest run
      - run: pnpm build
```

8. Modelo de dados (tipos centrais)

Definir bem os tipos antes de programar reduz retrabalho. O objeto `ProjectState` é tudo que precisa ser salvo: é serializado em JSON para arquivo e para o link compartilhável.

```
// src/core/types.ts
export type Vec2 = readonly [number, number];
export type Vec3 = readonly [number, number, number];

export interface Rect { xmin: number; xmax: number; ymin: number; ymax: number }
export interface Box  extends Rect { zmin: number; zmax: number }

/** Um objeto que aparece na lista de expressões (cada um tem cor e visibilidade). */
export type SceneObject =
  | SurfaceObj | DoubleIntegralObj | TripleIntegralObj
  | GradientObj | VectorFieldObj | OdeObj | ParametricObj | ImplicitObj;

interface BaseObj { id: string; label: string; color: string; visible: boolean }

export interface SurfaceObj extends BaseObj {
  kind: 'surface';
  expr: string;                        // "sin(x)*cos(y)"
  domain: Rect;
  resolution: [number, number];        // [nx, ny]
  colormap: 'viridis' | 'magma' | 'coolwarm' | 'height' | 'gradient-mag';
  opacity: number; wireframe: boolean; contourLines: number | null;
}

export type Region2D =
  | { type: 'rect';  rect: Rect }
  | { type: 'typeI'; a: string; b: string; g1: string; g2: string }   // x em [a,b], y em [g1(x), g2(x)]
  | { type: 'typeII'; c: string; d: string; h1: string; h2: string }  // y em [c,d], x em [h1(y), h2(y)]
  | { type: 'polar'; th1: string; th2: string; r1: string; r2: string }
  | { type: 'implicit'; inequalities: string[]; bounds: Rect };       // ["x^2+y^2<=4", "y>=x"]
export type RiemannRule = 'left' | 'right' | 'mid' | 'lower' | 'upper' | 'random';

export interface DoubleIntegralObj extends BaseObj {
  kind: 'double';
  surfaceId: string;                   // aponta para a SurfaceObj que é o integrando f
  region: Region2D;
  n: number; m: number;                // divisões em cada direção
  rule: RiemannRule;
  order: 'dydx' | 'dxdy';
  showSolid: boolean; density?: string; // ρ(x,y) opcional (massa, centroide)
}

export type Region3D =
  | { type: 'typeI3'; base: Region2D; u1: string; u2: string }        // z entre u1(x,y) e u2(x,y)
  | { type: 'cylindrical'; th: [string, string]; r: [string, string]; z: [string, string] }
  | { type: 'spherical';   th: [string, string]; ph: [string, string]; rho: [string, string] }
  | { type: 'implicit3'; inequalities: string[]; bounds: Box };

export interface TripleIntegralObj extends BaseObj {
  kind: 'triple'; integrand: string; region: Region3D;
  method: 'gauss' | 'montecarlo' | 'adaptive'; samples: number; sliceAxis?: 'x' | 'y' | 'z'; slicePos?: number;
}

export interface OdeObj extends BaseObj {
  kind: 'ode';
  system: string[];                    // ["sigma*(y-x)", "x*(rho-z)-y", "x*y-beta*z"]
  vars: string[];                      // ["x","y","z"]  (ou ["t","y"] se não autônomo)
  initial: number[][];                 // várias condições iniciais
  tspan: [number, number];
  method: 'euler' | 'heun' | 'rk4' | 'rk45';
  tol: number; step: number;
  params: Record<string, number>;
}

export interface ProjectState {
  version: 1;
  params: Record<string, number>;      // sliders globais: a, b, c, ...
  objects: SceneObject[];
  camera: { position: Vec3; target: Vec3; ortho: boolean };
  settings: { lang: 'pt-BR' | 'en'; theme: 'light' | 'dark'; sphericalConvention: 'physics' | 'math' };
}
```

Versionamento do arquivo: o campo `version` permite migrar projetos antigos quando o formato mudar. Toda mudança incompatível cria uma função `migrate_v1_to_v2` coberta por teste.

9. Motor de expressões e derivadas

9.1 Do texto à função rápida

Avaliar uma fórmula milhões de vezes (malhas, integrais, trajetórias) com um interpretador de AST é lento. A estratégia é: **parse → validar → gerar código JavaScript apenas com símbolos permitidos → compilar uma vez com** `new Function` **→ reutilizar**. Em geral isso é de 10 a 50 vezes mais rápido que avaliar a árvore a cada ponto.

> **Segurança**
> Nunca passe o texto do usuário diretamente para `new Function` ou `eval`. O gerador abaixo **só emite** números, variáveis declaradas, constantes e funções de uma lista branca. Qualquer outro nó da AST lança erro. Em projetos compartilhados por link, esse cuidado impede que um link malicioso execute código. Adicione também uma Content-Security-Policy no deploy.

```
// src/core/expr/codegen.ts  (versão simplificada)
import { parse, type MathNode } from 'mathjs';

const FN: Record<string, string> = {
  sin: 'Math.sin', cos: 'Math.cos', tan: 'Math.tan', asin: 'Math.asin', acos: 'Math.acos',
  atan: 'Math.atan', atan2: 'Math.atan2', sinh: 'Math.sinh', cosh: 'Math.cosh', tanh: 'Math.tanh',
  exp: 'Math.exp', log: 'Math.log', ln: 'Math.log', sqrt: 'Math.sqrt', abs: 'Math.abs',
  floor: 'Math.floor', min: 'Math.min', max: 'Math.max', pow: 'Math.pow',
};
const CONST: Record<string, string> = { pi: 'Math.PI', e: 'Math.E' };
const OPS = new Set(['+', '-', '*', '/', '^']);

function emit(n: MathNode, vars: ReadonlySet<string>): string {
  switch (n.type) {
    case 'ConstantNode':
      if (typeof n.value !== 'number') throw new Error('Constante inválida');
      return `(${n.value})`;
    case 'SymbolNode':
      if (vars.has(n.name)) return n.name;
      if (n.name in CONST) return CONST[n.name]!;
      throw new Error(`Símbolo desconhecido: ${n.name}`);
    case 'ParenthesisNode':
      return `(${emit(n.content, vars)})`;
    case 'OperatorNode': {
      if (!OPS.has(n.op)) throw new Error(`Operador não suportado: ${n.op}`);
      const a = n.args.map((x: MathNode) => emit(x, vars));
      if (a.length === 1) return `(${n.op}${a[0]})`;                // unário
      if (n.op === '^') return `Math.pow(${a[0]},${a[1]})`;
      return `(${a[0]}${n.op}${a[1]})`;
    }
    case 'FunctionNode': {
      const f = FN[n.fn.name];
      if (!f) throw new Error(`Função não suportada: ${n.fn.name}`);
      return `${f}(${n.args.map((x: MathNode) => emit(x, vars)).join(',')})`;
    }
    default:
      throw new Error(`Expressão não suportada (${n.type})`);
  }
}
/** compileExpr("x^2+a*y", ["x","y"], ["a"]) -> (x, y, p) => number, p = {a: 2} */
export function compileExpr(src: string, vars: string[], params: string[] = []) {
  const all = new Set([...vars, ...params]);
  const body = emit(parse(src), all);
  const unpack = params.length ? `const {${params.join(',')}} = p;` : '';
  return new Function(...vars, 'p', `${unpack} return ${body};`) as
    (...args: [...number[], Record<string, number>]) => number;
}
```

Detalhes que valem atenção:

- **Multiplicação implícita** ("2x", "3sin(x)", "xy"): o math.js já interpreta; teste bastante, pois "xy" pode significar a variável *xy* ou o produto x·y. Decida e documente (recomendado: variáveis têm uma letra).
- **Parâmetros livres:** a lista de símbolos que não são variáveis nem constantes vira automaticamente a lista de sliders. Valor inicial 1, faixa [−5, 5], editável.
- **Domínio:** `sqrt(-1)`, `log(0)` e `1/0` produzem NaN/Infinity. Trate como "buraco" na malha (não desenhe o triângulo) e avise no painel.
- **Mensagens de erro em português**, com a posição do problema ("parêntese não fechado na coluna 7").
- **Notação:** aceitar "sen" e "tg" (português) e "sin", "tan" (inglês) como sinônimos.

9.2 Derivadas e gradiente: três camadas

Camada	Como	Vantagem	Limitação
1. Simbólica	math.derivative(expr, "x") → simplifica → compila com compileExpr	Exata e rápida depois de compilada	Algumas funções (abs, floor, min, max) não derivam bem
2. Diferenciação automática (modo direto)	Números duais (valor, derivada) propagados na AST	Exata (precisão de máquina) para qualquer expressão da lista branca	Um passe por variável
3. Diferenças finitas centrais	(f(x+h) − f(x−h)) / 2h com h ≈ 1e-5·max(1,|x|)	Sempre funciona; ótima como verificação	Precisão ≈ 1e-10 no melhor caso

Estratégia: usar a camada 1; se falhar, a 2; e a camada 3 nos **testes** (cada gradiente simbólico é comparado numericamente com diferenças finitas em pontos aleatórios) e como último recurso em produção.

9.3 Regiões e pertinência

- Toda região 2D/3D implementa `contains(p)`, `bounds()` e, quando possível, `limits()` (limites analíticos para integração iterada).
- Regiões por **desigualdades** ($x^2 + y^2 \le 4$, $y \ge x$) são tratadas por varredura: para cada x, acha-se os intervalos de y onde todas as desigualdades valem, usando amostragem + bissecção nas fronteiras; o resultado alimenta a integração iterada. Se a região for desconexa, a integral é a soma dos intervalos.
- Quando a varredura não for confiável (fronteiras muito oscilantes), usar Monte Carlo/QMC dentro da caixa envolvente e exibir o erro estatístico estimado.

10. Motor numérico em detalhe

10.1 Quadratura de Gauss–Legendre

Para um integrando suave, n pontos de Gauss–Legendre integram exatamente polinômios até grau 2n − 1, o que faz converter muito mais rápido que Simpson ou trapézios. Os nós e pesos são calculados uma vez por n (cache) pelo método de Newton sobre os polinômios de Legendre.

```
// src/core/quad/gauss.ts
const cache = new Map<number, { x: Float64Array; w: Float64Array }>();

/** Nós e pesos de Gauss–Legendre em [-1, 1], ordenados de forma crescente. */
export function gaussLegendre(n: number) {
  const hit = cache.get(n);
  if (hit) return hit;
  const x = new Float64Array(n), w = new Float64Array(n);
  const m = (n + 1) >> 1;
  for (let i = 0; i < m; i++) {
    let z = Math.cos((Math.PI * (i + 0.75)) / (n + 0.5));   // chute inicial
    let pp = 1;
    for (let it = 0; it < 100; it++) {
      let p1 = 1, p2 = 0;
      for (let j = 0; j < n; j++) {                          // recorrência de Legendre
        const p3 = p2; p2 = p1;
        p1 = ((2 * j + 1) * z * p2 - j * p3) / (j + 1);
      }
      pp = (n * (z * p1 - p2)) / (z * z - 1);                // derivada P_n'(z)
      const dz = p1 / pp;
      z -= dz;
      if (Math.abs(dz) < 1e-15) break;
    }
    x[i] = -z; x[n - 1 - i] = z;
    w[i] = w[n - 1 - i] = 2 / ((1 - z * z) * pp * pp);
  }
  const out = { x, w };
  cache.set(n, out);
  return out;
}
```

10.2 Integral dupla iterada (região tipo I)

```
// src/core/quad/iterated.ts
type F2 = (x: number, y: number) => number;
type F1 = (x: number) => number;

/** ∫_a^b ∫_{g1(x)}^{g2(x)} f(x,y) dy dx  com Gauss–Legendre de n pontos em cada nível. */
export function integral2TypeI(f: F2, a: number, b: number, g1: F1, g2: F1, n = 32): number {
  const { x: t, w } = gaussLegendre(n);
  const cx = 0.5 * (b - a), mx = 0.5 * (b + a);
  let outer = 0;
  for (let i = 0; i < n; i++) {
    const x = cx * t[i]! + mx;
    const lo = g1(x), hi = g2(x);
    const cy = 0.5 * (hi - lo), my = 0.5 * (hi + lo);
    let inner = 0;
    for (let j = 0; j < n; j++) inner += w[j]! * f(x, cy * t[j]! + my);
    outer += w[i]! * cy * inner;
  }
  return cx * outer;
}
/** Polar: ∬ f(r cosθ, r senθ) r dr dθ */
export function integral2Polar(f: F2, th1: number, th2: number, r1: F1, r2: F1, n = 32) {
  return integral2TypeI((th, r) => f(r * Math.cos(th), r * Math.sin(th)) * r, th1, th2, r1, r2, n);
}
```

Observação importante: o integrando "enxerga" a fronteira apenas por meio dos limites g1 e g2. Se a função tiver uma quina (por exemplo $|x - y|$) *dentro* da região, Gauss–Legendre perde a ordem alta. Nesses casos, o app deve (1) detectar pelo estimador de erro (comparar n e 2n) e (2) dividir a região na quina ou cair para o método adaptativo G7K15.

10.3 Soma de Riemann para visualização

- Divide-se o retângulo envolvente em n × m células de tamanho Δx × Δy.
- Para cada célula escolhe-se o ponto de amostra pela regra (esquerda, direita, centro, ...). Para as somas inferior/superior, avalia-se o mínimo/máximo na célula (amostrando 5×5 pontos e, se disponível, usando o gradiente para refinar).
- Em regiões não retangulares: **modo simples** usa apenas células cujo centro pertence à região; **modo preciso** pondera cada célula pela fração de área dentro da região (estimada por subamostragem). O modo é indicado na interface para o aluno entender de onde vem o erro.
- O erro mostrado ao usuário é $|S_{n,m} - I|$, em que I vem da quadratura de alta precisão; o gráfico de convergência calcula S para n = 2, 4, 8, ..., 256 e estima a ordem pela inclinação em log-log.

10.4 Integral tripla

```
// ∭ f dV com E = {(x,y) ∈ D, u1(x,y) ≤ z ≤ u2(x,y)} , D tipo I
export function integral3TypeI(f: F3, a: number, b: number, g1: F1, g2: F1,
                               u1: F2, u2: F2, n = 24): number {
  const { x: t, w } = gaussLegendre(n);
  const cx = 0.5 * (b - a), mx = 0.5 * (b + a);
  let sx = 0;
  for (let i = 0; i < n; i++) {
    const x = cx * t[i]! + mx;
    const lo = g1(x), hi = g2(x), cy = 0.5 * (hi - lo), my = 0.5 * (hi + lo);
    let sy = 0;
    for (let j = 0; j < n; j++) {
      const y = cy * t[j]! + my;
      const zl = u1(x, y), zh = u2(x, y), cz = 0.5 * (zh - zl), mz = 0.5 * (zh + zl);
      let sz = 0;
      for (let k = 0; k < n; k++) sz += w[k]! * f(x, y, cz * t[k]! + mz);
      sy += w[j]! * cz * sz;
    }
    sx += w[i]! * cy * sy;
  }
  return cx * sx;
}
```

- **Custo:** n^3 avaliações (n = 24 → 13.824; n = 48 → 110.592). Rápido em Worker; use n adaptativo por tentativa: calcula com n e 2n e aceita se a diferença relativa for menor que a tolerância.
- **Cilíndricas e esféricas:** o app aplica o Jacobiano (r ou $\rho^2 \operatorname{sen} \varphi$) ao integrando e integra em uma caixa de parâmetros; o desenho do sólido usa a transformação inversa.
- **Monte Carlo/QMC:** sequência de Halton ou Sobol dentro da caixa envolvente, contando apenas pontos dentro da região: $I \approx V_{caixa} \cdot \text{média}(f \cdot 1_E)$; erro estimado pelo desvio padrão. Executado em lotes de 50 mil pontos para a UI mostrar progresso e permitir cancelar.
- **Cubatura adaptativa (Genz–Malik):** subdivide o hipercubo onde o erro estimado é maior; é a opção para integrandos com picos.

10.5 EDOs

```
// src/core/ode/rk4.ts  (passo fixo, sistema de dimensão d)
export type Rhs = (t: number, y: Float64Array, dy: Float64Array) => void;

export function rk4(f: Rhs, t0: number, y0: Float64Array, h: number, steps: number) {
  const d = y0.length;
  const y = Float64Array.from(y0);
  const k1 = new Float64Array(d), k2 = new Float64Array(d);
  const k3 = new Float64Array(d), k4 = new Float64Array(d), tmp = new Float64Array(d);
  const T = new Float64Array(steps + 1);
  const Y = new Float64Array((steps + 1) * d);          // trajetória achatada: Y[s*d + i]
  T[0] = t0; Y.set(y, 0);
  for (let s = 0; s < steps; s++) {
    const t = t0 + s * h;
    f(t, y, k1);
    for (let i = 0; i < d; i++) tmp[i] = y[i]! + 0.5 * h * k1[i]!;
    f(t + 0.5 * h, tmp, k2);
    for (let i = 0; i < d; i++) tmp[i] = y[i]! + 0.5 * h * k2[i]!;
    f(t + 0.5 * h, tmp, k3);
    for (let i = 0; i < d; i++) tmp[i] = y[i]! + h * k3[i]!;
    f(t + h, tmp, k4);
    for (let i = 0; i < d; i++) y[i] = y[i]! + (h / 6) * (k1[i]! + 2 * k2[i]! + 2 * k3[i]! + k4[i]!);
    T[s + 1] = t + h; Y.set(y, (s + 1) * d);
  }
  return { T, Y, d };
}
```

- **RK45 (Dormand–Prince):** estima o erro local com a diferença entre as soluções de ordem 4 e 5; o passo é ajustado por $h_{novo} = 0{,}9 \cdot h \cdot (\text{tol}/\text{erro})^{1/5}$, limitado a [0,2; 5]·h. Guarde a saída densa (interpolação) para desenhar curvas suaves e animar sem pular passos.
- **Eventos e segurança:** parar quando |y| > 1e6 (explosão em tempo finito), quando a trajetória sai do domínio visível, ou quando o número máximo de passos é atingido; avisar o usuário.
- **Campo de direções:** para y' = f(t, y), em cada ponto da malha desenha-se um segmento de comprimento fixo na direção (1, f); instanciado na GPU para suportar 40×40 ou mais.
- **Equilíbrios:** Newton a partir de uma grade de sementes; remover duplicatas por distância; calcular a Jacobiana (por derivada simbólica ou diferenças finitas) e os autovalores. Em 2D, use tr(J) e det(J): det < 0 → sela; det > 0 e $\text{tr}^2 - 4\det \geq 0$ → nó (estável se tr < 0); det > 0 e $\text{tr}^2 - 4\det < 0$ → foco (estável se tr < 0) ou centro (tr = 0).
- **Comparação de métodos:** executar Euler, Heun e RK4 com o mesmo h e plotar o erro contra a solução exata (se informada) ou contra uma referência RK45 de alta precisão.

11. Renderização 3D com Three.js

11.1 Cena, câmera e convenções

- **Z para cima.** Three.js usa Y para cima por padrão; para Matemática o correto é Z. Defina `camera.up.set(0, 0, 1)` e configure os controles com esse eixo. Todo o código deve tratar (x, y, z) como na matemática.
- **Perspectiva e ortográfica:** alternância por botão (a ortográfica é melhor para comparar tamanhos e ler valores).
- **Controles:** órbita (botão esquerdo), pan (direito), zoom (roda), duplo clique para centralizar no ponto. Botões para vistas padrão: topo (plano xy), frente, lateral e isométrica.
- **Escala:** permitir razões de aspecto independentes por eixo (útil quando z varia pouco ou muito), com a opção "1:1:1" para geometria verdadeira.
- **Eixos com ticks:** algoritmo de "números bonitos" (1, 2, 5 × 10^k) para escolher o espaçamento; rótulos por sprites/texto SDF (ex.: troika-three-text) para ficarem nítidos em qualquer zoom.
- **Renderização sob demanda:** um `requestRender()` marca a cena como suja; o loop só desenha quando necessário ou durante animações.

11.2 Superfície z = f(x, y)

- Malha regular nx × ny: `positions` (Float32Array de 3·nx·ny) e índices de triângulos (Uint32Array).
- **Normais analíticas** a partir do gradiente: n = (−f_x, −f_y, 1)/‖·‖. São mais suaves e mais baratas que `computeVertexNormals()` e valem para iluminação correta.
- **Cor por shader:** o atributo de valor (altura, |∇f|, curvatura) vai como `attribute float`; o fragment shader consulta uma textura 1D de paleta (colormap).
- **Curvas de nível no shader:** ao desenhar, calcula-se `float k = fract(v / step)` e suaviza-se a linha com `fwidth` para ficar com espessura constante na tela.
- **Buracos e descontinuidades:** triângulos com vértice NaN/Infinity ou salto maior que um limite relativo são descartados (isso evita "paredes" falsas em tan(x), 1/x).
- **Transparência:** `depthWrite: false` com ordenação por objeto resolve a maioria dos casos; para cenas complexas, implementar *weighted blended OIT* (Fase 8).

11.3 Prismas de Riemann com InstancedMesh

Uma malha de 100 × 100 já são 10.000 caixas. Criar 10.000 `Mesh` destruiria o desempenho; a solução é **uma única InstancedMesh**, em que cada instância tem sua matriz (posição e escala) e cor.

```
// src/render/objects/riemannObject.ts
import * as THREE from 'three';

export function buildRiemannColumns(heights: Float32Array, nx: number, ny: number,
                                    x0: number, y0: number, dx: number, dy: number) {
  const geo = new THREE.BoxGeometry(1, 1, 1);
  geo.translate(0, 0, 0.5);                          // base em z=0, cresce para +z (Z para cima)
  const mat = new THREE.MeshStandardMaterial({ transparent: true, opacity: 0.6, side: THREE.DoubleSide });
  const mesh = new THREE.InstancedMesh(geo, mat, nx * ny);
```

```
  const m = new THREE.Matrix4(), pos = new THREE.Vector3(), q = new THREE.Quaternion();
  const sc = new THREE.Vector3(), col = new THREE.Color();
  let hmax = 1e-12;
  for (let q2 = 0; q2 < heights.length; q2++) hmax = Math.max(hmax, Math.abs(heights[q2]!));
  let k = 0;
  for (let j = 0; j < ny; j++) {
    for (let i = 0; i < nx; i++, k++) {
      const h = heights[k]!;
      pos.set(x0 + (i + 0.5) * dx, y0 + (j + 0.5) * dy, 0);
      sc.set(dx * 0.98, dy * 0.98, h === 0 ? 1e-6 : h);   // escala z negativa inverte a caixa p/ baixo
      m.compose(pos, q, sc);
      mesh.setMatrixAt(k, m);
      mesh.setColorAt(k, col.setHSL(0.66 * (1 - Math.min(1, Math.abs(h) / hmax)), 0.7, 0.5));
    }
  }
  mesh.instanceMatrix.needsUpdate = true;
  if (mesh.instanceColor) mesh.instanceColor.needsUpdate = true;
  return mesh;
}
```

- Escala z negativa inverte a orientação das faces; por isso `DoubleSide`.
- Ao mexer no slider de n, **reutilize** a InstancedMesh quando a contagem couber (mude `mesh.count`) e recrie só se exceder a capacidade.
- Acima de ~250 mil prismas, trocar para uma superfície em degraus gerada em shader (ou mostrar só a silhueta).

11.4 Sólidos, fatias e nuvens de pontos

- **Regiões tipo 1:** o sólido é fechado por três malhas: a superfície inferior z = u1(x, y), a superior z = u2(x, y) e as "paredes" laterais definidas pela fronteira de D. Cada uma é uma malha paramétrica.
- **Regiões implícitas:** marching cubes sobre uma grade 3D (em Worker); malha resultante com normais pelo gradiente da função implícita.
- **Corte:** `material.clippingPlanes` com um plano controlado por slider; para mostrar a "tampa" do corte, usar a técnica de stencil buffer (Fase 5, opcional).
- **Fatias:** para cada z_k desenha-se a seção transversal (contorno da região D_k com preenchimento), animada de z_{min} a z_{max}.
- **Monte Carlo:** `THREE.Points` com cor por valor e tamanho em pixels; até alguns milhões de pontos se mantém fluido.

11.5 Vetores, setas e trilhas

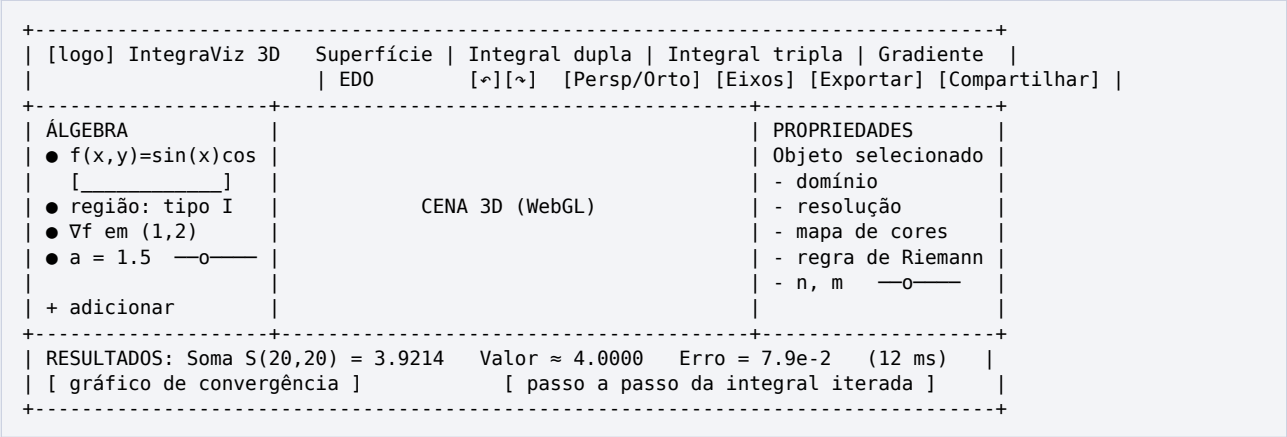
- **Campo de gradientes / direções:** setas instanciadas (cone + cilindro em uma única InstancedMesh por tipo). Opções: comprimento proporcional ao módulo, normalizado (só direção) ou saturado; cor pelo módulo.
- **Linhas espessas:** `Line2`/`LineMaterial` (examples/jsm/lines), pois linhas WebGL nativas têm sempre 1 px. Usadas em curvas de nível, trajetórias e contornos de região.
- **Trilhas animadas de EDO:** buffer pré-alocado e `setDrawRange` crescente; esmaecer a cauda por atributo de alfa. Partícula-guia como esfera instanciada.
- **Picking:** `Raycaster` contra a superfície devolve (x, y, z); a UI mostra f, ∇f e permite "fixar" o ponto. Em campos de direções, o clique vira condição inicial.

11.6 Desempenho de renderização

Técnica	Quando aplicar
Resolução adaptativa	Durante rotação/arrasto usar malha 1/2 ou 1/4 e refinar ao soltar (≈150 ms).
InstancedMesh / merge	Qualquer conjunto de objetos repetidos (prismas, setas, pontos).
Reuso de buffers	Atualizar `attribute.array` e `needsUpdate` em vez de recriar geometrias.
Dispose	Sempre chamar `geometry.dispose()`, `material.dispose()` e liberar texturas ao remover objetos.
Pixel ratio limitado	`renderer.setPixelRatio(Math.min(devicePixelRatio, 2))`.
Frustum culling	Ativado por padrão; manter bounding spheres corretas após atualizar buffers.
Renderização sob demanda	Economiza bateria e deixa o notebook frio durante a aula.
OffscreenCanvas (avançado)	Mover o render para um Worker se a thread principal ficar disputada.

12. Projeto da interface (UX)

12.1 Layout



12.2 Painel de expressões (coração da interface)

- Cada linha é um objeto: bolinha de cor (liga/desliga visibilidade), campo de fórmula (MathLive) e menu de contexto.
- O campo aceita **LaTeX visual** (frações, raízes, expoentes) e **texto** ("x^2 + y^2", "sqrt(x)"). Internamente tudo é convertido para a sintaxe do motor de expressões.
- Erros aparecem abaixo do campo, em vermelho, com sugestão ("Você quis dizer sin(x)?").
- Parâmetros livres geram **sliders** automaticamente, com botão de animação (reproduzir ▶).
- Arrastar para reordenar; duplicar; agrupar objetos por tema.
- Atalhos: `Ctrl+Z / Ctrl+Y`, `Ctrl+Enter` (adicionar), `Espaço` (animar), `R` (resetar câmera), `G` (grade).

12.3 Fluxos de uso principais

Fluxo 1 — Integral dupla sobre um retângulo

1. Clicar em "Integral dupla" e digitar `f(x,y) = x*y`.
2. Região retangular padrão [0, 1] × [0, 1]; a superfície aparece com a malha.
3. Ativar "Prismas de Riemann"; escolher "ponto médio"; mover n de 4 até 60.
4. Ver no painel inferior: S(n) tende a 0,25; erro cai com ordem 2 no gráfico log-log.
5. Ativar "Sólido" para ver o volume exato; alternar "Mudar ordem" para ver dx dy ↔ dy dx.

Fluxo 2 — Região tipo I

1. Digitar o integrando; escolher "Tipo I"; informar a = 0, b = 1, g1 = x^2, g2 = √x.
2. O plano xy mostra a região entre as curvas; os prismas só aparecem dentro dela.
3. O painel exibe a integral iterada montada em LaTeX com os limites substituídos.
4. Botão "Converter para tipo II": o app calcula os limites inversos (h1(y) = y^2, h2(y) = √y para este exemplo) e compara.

Fluxo 3 — Esfera em coordenadas esféricas

1. "Integral tripla" → coordenadas esféricas; integrando 1; $\rho \in [0, 1]$, $\varphi \in [0, \pi]$, $\theta \in [0, 2\pi]$.
2. O app mostra o sólido, o Jacobiano $\rho^2 \operatorname{sen} \varphi$ e o resultado 4,18879 ≈ 4π/3.
3. Slider de φ máximo transforma a esfera em cone/calota; o volume atualiza.

Fluxo 4 — EDO de Lorenz

1. "EDO" → sistema 3D; escolher o preset "Lorenz" ($\sigma = 10$, $\rho = 28$, $\beta = 8/3$).
2. Adicionar duas condições iniciais muito próximas; ver a divergência exponencial no gráfico de distância.
3. Mover ρ de 10 até 30 e observar a mudança de equilíbrio estável para o atrator caótico.

12.4 Acessibilidade, internacionalização e responsividade

- **Teclado:** todos os controles alcançáveis por Tab; a cena responde a setas (rotacionar) e +/− (zoom).
- **Contraste e daltonismo:** paletas perceptualmente uniformes (viridis) como padrão; evitar depender só de vermelho/verde.
- **Redução de movimento:** respeitar `prefers-reduced-motion` (desligar animações automáticas).
- **Textos alternativos e leitores de tela:** resultados numéricos em região `aria-live`.
- **i18n:** pt-BR como padrão e en; separar textos em arquivos JSON desde o início (retrofit é doloroso).
- **Telas pequenas:** painéis recolhíveis; gestos de pinça/arrasto; botões de tamanho adequado ao toque.

13. Roadmap de execução por fases

O cronograma abaixo assume **uma pessoa** dedicando de 10 a 15 horas por semana. Com dedicação integral (30 a 40 h/semana), divida os prazos por 2 ou 3. As fases foram desenhadas para que **cada uma termine com algo utilizável e publicado**, evitando o risco de passar meses sem uma versão funcional.

Fase	Nome	Duração	Entregável principal	Critério de aceite
0	Preparação	1 semana	Repositório, CI, protótipo "hello surface"	pnpm dev mostra z = sen(x)cos(y) girando; CI verde
1	Fundação e superfícies	2 semanas	Cena 3D + parser + superfície com sliders	Fórmula digitada vira superfície em < 100 ms (128×128)
2	Integral dupla retangular	3 semanas	Prismas de Riemann, valor, erro, convergência	Casos de teste A1–A4 (Seção 14) passam
3	Regiões gerais e polar	3 semanas	Tipo I/II, polar, desigualdades, mudar ordem	A5–A9 passam; conversão I↔II correta
4	Diferencial multivariável	2 semanas	Gradiente, direcional, plano tangente, pontos críticos	D1–D4 passam; setas fluidas (60 fps)
5	Integral tripla	4 semanas	Sólidos, cilíndricas, esféricas, fatias, Monte Carlo	T1–T7 passam; esfera = 4π/3 (erro < 1e-6)
6	EDOs	4 semanas	Campo de direções, RK4/RK45, fase 2D, 3D, equilíbrios	E1–E6 passam; ordem de RK4 ≈ 4
7	Plataforma e polimento	3 semanas	Salvar/abrir/link, exportar, i18n, exemplos, tutorial	Abrir link reproduz a cena; auditoria de acessibilidade
8	Desempenho e WASM	3 semanas	Profiling, LOD, Rust→WASM nos pontos quentes	Ganho ≥ 3× nos benchmarks escolhidos
9	Extras (contínuo)	livre	Valores exatos (SymPy/Pyodide), EDPs, integrais de linha/superfície	Definido caso a caso

Total estimado: cerca de 22 a 26 semanas até a Fase 8, com o **MVP na semana 8 ou 9** (fim da Fase 3).

Fase 0 — Preparação (1 semana)

- ☐ Criar repositório, rodar o scaffolding do Vite, configurar ESLint, Prettier, Husky e CI (Seção 7).
- ☐ Escrever três ADRs (Architecture Decision Records) curtos em `docs/adr/`: (1) linguagem e stack; (2) convenções matemáticas: eixo Z para cima, ângulos esféricos, notação; (3) formato do estado/arquivo.
- ☐ Estudar o básico de Three.js (cena, câmera, renderer, geometria, material, luzes, controles).
- ☐ Protótipo descartável: superfície fixa z = sen(x)cos(y) em malha 64×64, com OrbitControls.
- ☐ Ler a documentação de parse/derivative do math.js e testar 10 expressões no console.

Fase 1 — Fundação e superfícies (2 semanas)

Semana 1

- ☐ Layout de 3 colunas (Álgebra | Cena | Propriedades) com Tailwind.
- ☐ Módulo `render/scene.ts`: renderer, câmera com `up = (0,0,1)`, controles, resize, render sob demanda.
- ☐ Eixos com ticks "bonitos" e rótulos; planos de grade.
- ☐ `core/expr/codegen.ts` (Seção 9) com 20 testes unitários (válidos e inválidos).
- ☐ Store Zustand com `ProjectState` inicial e um `Surface0bj`.
- ☐ Campo de fórmula simples (texto) com mensagens de erro em português.

Semana 2

- ☐ Worker de cálculo + Comlink; debounce 120 ms; cancelamento por id de job.
- ☐ Construtor de malha de superfície com normais analíticas e tratamento de NaN/Infinity.
- ☐ Sliders automáticos para parâmetros livres; animação de parâmetro.
- ☐ Mapas de cores por textura 1D (viridis, magma, coolwarm); curvas de nível no shader.
- ☐ Picking: clicar na superfície mostra (x, y, f).
- ☐ Primeiro deploy público (preview) e GIF de demonstração para o README.

> **Aceite da Fase 1**
> Digitar `sin(a*x)*cos(y)`, ver o slider "a" aparecer e a superfície mudar suavemente enquanto ele é arrastado, sem travar a interface; digitar `sin(` mostra erro claro e mantém a última superfície válida na tela.

Fase 2 — Integral dupla retangular (3 semanas)

Semana 3 — núcleo numérico

- ☐ `core/quad/gauss.ts` e `iterated.ts` (Seção 10).
- ☐ Regras de Riemann: esquerda, direita, ponto médio, inferior, superior, aleatória.
- ☐ Testes com valores exatos (A1–A4) e testes de ordem de convergência (médio ≈ 2, esquerda ≈ 1).

Semana 4 — visual e interface

- ☐ InstancedMesh de prismas (Seção 11.3) com cor por altura; sliders de n e m (2 a 200).
- ☐ Painel de resultados: S(n, m), valor de alta precisão, erro absoluto e relativo, tempo.
- ☐ Realce da célula sob o cursor mostrando f(x*, y*), ΔA e a contribuição à soma.

Semana 5 — convergência e sólido

- ☐ Gráfico log-log do erro × n com inclinação estimada ("ordem ≈ 2,0").
- ☐ Sólido translúcido do volume exato; alternância prismas/sólido/ambos.
- ☐ Exemplos prontos (Apêndice B) e documentação do módulo.

Fase 3 — Regiões gerais e polar (3 semanas)

- ☐ `core/regions`: tipo I, tipo II, polar; `contains()` e `bounds()`.
- ☐ Editor de região: campos de limites com validação (por exemplo, g1 ≤ g2 em todo o intervalo; aviso se cruzar).
- ☐ Desenho da região no plano xy e "subida" dos prismas apenas dentro dela (modos simples e preciso).
- ☐ Polar: setores com área r dr dθ; animação mostrando por que o Jacobiano é r.
- ☐ Regiões por desigualdades (varredura + bissecção, com fallback Monte Carlo).
- ☐ **Mudar a ordem de integração**: conversão tipo I ↔ tipo II por amostragem das fronteiras e identificação de ramos; quando não for possível automaticamente, avisar e permitir informar os limites manualmente.
- ☐ Aplicações: massa, centro de massa, momentos de inércia, valor médio, área de superfície.
- ☐ Painel "passo a passo" com a integral iterada em LaTeX.

Marco: MVP publicado
Ao fim desta fase, publicar a versão 0.3 com um post/vídeo curto. Coletar feedback de 3 a 5 pessoas (colegas, professor, monitor) antes de começar o módulo 3D completo: o que elas tentaram digitar e não funcionou é o melhor guia para a próxima prioridade.

Fase 4 — Cálculo diferencial multivariável (2 semanas)

- ☐ `core/diff`: gradiente (simbólico + autodiff + diferenças finitas de verificação), Hessiana.
- ☐ Ponto arrastável sobre a superfície; vetor gradiente 3D e sua projeção no plano; campo de gradientes com setas instanciadas.
- ☐ Derivada direcional: círculo unitário para escolher u; reta tangente; valor $D_u f$.
- ☐ Plano tangente e linearização com slider de vizinhança (mostrar o erro da aproximação).
- ☐ Pontos críticos: Newton com sementes; classificação por Hessiana; marcadores coloridos.
- ☐ Descida do gradiente animada com histórico de passos e gráfico de $f(p_k)$.

Fase 5 — Integral tripla (4 semanas)

- ☐ `core/regions/region3d.ts` e integral tripla iterada (Seção 10.4).
- ☐ Cilíndricas e esféricas com Jacobianos e diagrama de convenção de ângulos.
- ☐ Malhas de contorno do sólido (superfícies inferior/superior/paredes) e marching cubes para regiões implícitas.
- ☐ Corte por plano, fatias paralelas e animação de Cavalieri.
- ☐ Monte Carlo/QMC em lotes com barra de progresso, cancelamento e estimativa de erro.
- ☐ Presets: esfera, cone, paraboloide, toro, cilindros que se cruzam (volume de Steinmetz).
- ☐ Massa, centro de massa e momentos de inércia 3D com densidade variável.

Fase 6 — Equações diferenciais (4 semanas)

- ☐ `core/ode`: Euler, Heun, RK4, RK45 com saída densa, eventos e limites de segurança.
- ☐ Campo de direções 1ª ordem; clique cria solução; várias soluções simultâneas.
- ☐ Sistemas 2D: retrato de fase, nuclinas, equilíbrios, Jacobiana e classificação automática.
- ☐ Sistemas 3D com trilha animada (Lorenz, Rössler) e sliders de parâmetros.
- ☐ Conversão automática de EDO de 2ª ordem para sistema; gráficos y(t), y'(t).
- ☐ Comparação de métodos e erro contra solução exata/referência; tabela de ordem observada.
- ☐ Presets: oscilador, pêndulo, Van der Pol, Lotka–Volterra, logístico.

Fase 7 — Plataforma e polimento (3 semanas)

- ☐ Salvar/abrir JSON com validação de esquema e migrações; link compartilhável (hash comprimido).
- ☐ Exportar PNG, WebM (MediaRecorder) e GLTF.
- ☐ Biblioteca de exemplos com busca; tutorial guiado de 5 passos na primeira visita.
- ☐ i18n pt-BR/en completo; auditoria de acessibilidade (axe, navegação por teclado).
- ☐ Página "Sobre", licença, README com GIFs, guia de contribuição.
- ☐ Testes E2E Playwright dos 6 fluxos principais.

Fase 8 — Desempenho e WebAssembly (3 semanas)

- ☐ Perfil com Chrome DevTools: tempo de script, GC, GPU; registrar baselines em `docs/perf.md`.
- ☐ Benchmarks Vitest (`vitest bench`) para quadratura, malhas e EDO.
- ☐ Migrar para Rust/WASM apenas os laços que, medidos, forem gargalo (integral tripla, marching cubes, Monte Carlo).
- ☐ SharedArrayBuffer + múltiplos Workers (exige cabeçalhos COOP/COEP no deploy).
- ☐ OIT para transparência, LOD de malha e culling de instâncias.
- ☐ Opcional e exploratório: WebGPU compute para Monte Carlo e EDO massivas.

Fase 9 — Extras contínuos

- **Valor exato:** carregar Pyodide sob demanda, rodar SymPy (`integrate`) para integrais simbólicas e exibir o resultado em LaTeX ao lado do numérico; se SymPy não resolver, mostrar só o numérico.
- **EDPs:** calor e onda como superfícies animadas z = u(x, t).
- **Integrais de linha e de superfície** e visualização dos teoremas de Green, Stokes e Gauss.

Definição de "pronto" para cada funcionalidade

- Tipos TypeScript completos e sem any.
- Testes unitários (valores exatos ou propriedades) e, quando houver UI, um teste E2E.
- Textos em i18n; mensagens de erro compreensíveis.
- Dentro do orçamento de desempenho (Seção 15).
- Acessível por teclado e sem erros/avisos no console.
- Exemplo na biblioteca e nota curta em `docs/`.

14. Estratégia de testes e validação

Em software matemático, o maior risco é o **erro silencioso**: o gráfico bonito com o número errado. A estratégia de testes tem quatro camadas, e a primeira (matemática) é a mais importante.

14.1 Pirâmide de testes

Camada	Ferramenta	O que verifica
Matemática (unitária)	Vitest	Valores exatos conhecidos, ordem de convergência, gradientes vs diferenças finitas.
Propriedades	fast-check	Identidades que valem para entradas aleatórias (linearidade, Fubini, polar = cartesiano).
Integração	Vitest + jsdom	Worker ↔ estado ↔ geometria; formato JSON e migrações.
E2E e visual	Playwright	Fluxos de usuário; capturas de tela de cenas fixas com tolerância.
Desempenho	vitest bench	Tempos de quadratura, malha e EDO contra orçamentos.

14.2 Casos de referência com valor exato

Estes casos entram em `tests/fixtures/`. Para cada um, o teste exige erro relativo menor que a tolerância indicada.

ID	Integral	Região	Valor exato	Tol.
A1	$\iint x \cdot y \, dA$	$[0,1] \times [0,1]$	$1/4 = 0{,}25$	1e-12
A2	$\iint (x^2 + y^2) \, dA$	$[0,1] \times [0,1]$	$2/3$	1e-12
A3	$\iint \operatorname{sen} x \cdot \operatorname{sen} y \, dA$	$[0,\pi] \times [0,\pi]$	4	1e-10
A4	$\iint (x^2 + y^2) \, dA$	disco $x^2 + y^2 \le 1$ (polar)	$\pi/2 \approx 1{,}5707963$	1e-10
A5	$\iint x \, dA$	triângulo $0 \le y \le x \le 1$ (tipo I)	$1/3$	1e-12
A6	$\iint (x + y) \, dA$	entre $y = x^2$ e $y = \sqrt{x}$ (tipo I)	$3/10$	1e-12
A7	Volume sob $z = 1 - x^2 - y^2$	disco unitário	$\pi/2$	1e-10
A8	$\iint e^{-(x^2+y^2)} \, dA$	disco de raio 2 (polar)	$\pi(1 - e^{-4}) \approx 3{,}08405$	1e-10
A9	$\iint x \, dA$	disco unitário (simetria)	0	1e-12 (abs.)
T1	$\iiint x \cdot y \cdot z \, dV$	$[0,1]^3$	$1/8$	1e-12
T2	$\iiint 1 \, dV$ (volume da esfera)	esfera unitária (esféricas)	$4\pi/3 \approx 4{,}18879$	1e-8
T3	$\iiint 1 \, dV$	cone $z \ge \sqrt{(x^2 + y^2)}$, $z \le 1$ (cilíndricas)	$\pi/3 \approx 1{,}04720$	1e-8
T4	$\iiint z \, dV$	hemisfério superior unitário	$\pi/4 \approx 0{,}785398$	1e-8
T5	$\iiint (x^2 + y^2) \, dV$	esfera unitária	$8\pi/15 \approx 1{,}675516$	1e-8
T6	Volume do toro ($R = 2$, $r = 1$)	paramétrico/cilíndricas	$2\pi^2 R r^2 = 4\pi^2 \approx 39{,}4784$	1e-6
T7	Volume de Steinmetz (cilindros se cruzando, raio 1)	$x^2 + y^2 \le 1$, $x^2 + z^2 \le 1$	$16/3$	1e-6 (Monte Carlo: 1e-3)

ID	Integral	Região	Valor exato	Tol.
D1	∇f em (1, 2) para $f = x^2 + y^2$	—	(2, 4)	exato
D2	$D_u f$ com u = (3/5, 4/5)	mesmo f e ponto	4,4	1e-12
D3	Pontos críticos de $x^3 - 3x + y^2$	—	(1,0) mínimo; (−1,0) sela	exato
D4	Plano tangente de $z = x^2 + y^2$ em (1, 1)	—	$z = 2x + 2y - 2$	exato
E1	$y' = y$, $y(0) = 1$	$t \in [0, 1]$	$y(1) = e \approx 2{,}718282$	RK4 h=0,01: 1e-9
E2	$y'' + y = 0$, $y(0) = 1$, $y'(0) = 0$	$t \in [0, 2\pi]$	$y = \cos t$	RK4/RK45
E3	$y' = y(1 - y)$, $y(0) = 0{,}1$	$t \geq 0$	$y = 1/(1 + 9e^{-t})$	RK45 tol 1e-8
E4	$x' = y$, $y' = -x$	plano de fase	círculos; autovalores ±i (centro)	classificação
E5	Lorenz σ=10, ρ=28, β=8/3	equilíbrios	(0,0,0) e (±√72, ±√72, 27)	1e-8
E6	Sensibilidade: duas condições iniciais a 1e-8	Lorenz	distância cresce exponencialmente	qualitativo

Observação sobre T7: o volume da interseção de dois cilindros perpendiculares de raio r é $16r^3/3$; com r = 1 vale 16/3 ≈ 5,3333.

14.3 Testes de propriedades (fast-check)

- **Linearidade:** $I(\alpha f + \beta g) = \alpha I(f) + \beta I(g)$ em regiões aleatórias.
- **Fubini:** em retângulos, dy dx e dx dy dão o mesmo valor (dentro da tolerância).
- **Polar = cartesiano:** integral sobre discos e setores calculada nos dois sistemas.
- **Aditividade:** $I(D) = I(D_1) + I(D_2)$ ao dividir D por uma reta.
- **Simetria:** funções ímpares sobre regiões simétricas dão zero.
- **Mudança de ordem:** tipo I e tipo II da mesma região coincidem.
- **Gradiente:** o gradiente simbólico concorda com diferenças finitas centrais em pontos aleatórios (|erro| < 1e-6).
- **Parser:** expressões geradas aleatoriamente (fuzzing) nunca executam código fora da lista branca.

14.4 Ordem de convergência

Método	Razão esperada do erro ao dobrar n (ou dividir h por 2)	Teste
Riemann esquerda/direita	≈ 2 (ordem 1)	razão ∈ [1,8; 2,2]
Riemann ponto médio	≈ 4 (ordem 2)	razão ∈ [3,6; 4,4]
Euler	≈ 2	razão ∈ [1,8; 2,2]
Heun	≈ 4	razão ∈ [3,6; 4,4]
RK4	≈ 16	razão ∈ [14; 18] (antes de atingir o ruído de arredondamento)
Monte Carlo	≈ √2 ao dobrar N	em média sobre várias sementes

14.5 Testes visuais

- Cenas determinísticas (câmera, tamanho de janela e sementes fixos) com captura Playwright.
- Tolerância de diferença de pixels pequena; rodar no CI com renderização por software (SwiftShader) para reduzir variação de GPU.
- Testes visuais cobrem regressões grosseiras (objeto sumiu, cor errada); a correção numérica é garantida pelas camadas matemáticas.

14.6 Validação cruzada com Python

Para casos novos ou complexos, gere o gabarito com SymPy/SciPy em um script à parte (`tools/gabarito.py`), salve o resultado em JSON e use-o como fixture. Assim, o app é validado contra uma implementação independente, e o Python permanece como ferramenta de apoio, sem entrar no produto final.

```
# tools/gabarito.py (exemplo)
import sympy as sp
x, y = sp.symbols('x y', real=True)
print(sp.integrate(sp.integrate(x + y, (y, x**2, sp.sqrt(x))), (x, 0, 1)))   # 3/10

from scipy import integrate
import numpy as np
val, err = integrate.dblquad(lambda y, x: np.exp(-(x*x + y*y)), -2, 2,
                             lambda x: -np.sqrt(4 - x*x), lambda x: np.sqrt(4 - x*x))
print(val, err)
```

15. Desempenho: orçamentos e técnicas

Orçamentos-alvo (devem ser **validados com benchmarks na sua máquina** e ajustados; servem para detectar regressões):

Operação	Orçamento	Onde roda
Parse + compilação de uma fórmula	< 5 ms	Worker
Malha de superfície 128 × 128 (posições + normais + valores)	< 20 ms	Worker
Malha de superfície 256 × 256	< 60 ms	Worker
Atualizar 10.000 prismas (100 × 100)	< 30 ms	Worker + GPU
Integral dupla Gauss–Legendre 32 × 32	< 5 ms	Worker
Integral tripla Gauss–Legendre 24^3	< 150 ms (< 40 ms com WASM)	Worker
Monte Carlo 1.000.000 de pontos	< 1 s em lotes, UI responsiva	Worker
EDO RK4 3D, 100.000 passos	< 100 ms	Worker
Quadro de renderização	16,7 ms com até ~300 mil triângulos	GPU/main
Memória total	< 300 MB em cenas típicas	—

15.1 Técnicas de otimização de laços numéricos

- Usar `Float64Array`/`Float32Array` e pré-alocar; evitar criar objetos/arrays por ponto.
- Funções *monomórficas* (sempre os mesmos tipos de argumentos) ajudam o JIT do V8.
- Hoistar cálculos invariantes para fora dos laços internos (limites, constantes de mapeamento).
- Evitar closures novos dentro do laço; passar funções compiladas já prontas.
- Cache de nós/pesos de Gauss e de funções compiladas (chave = expressão + parâmetros).
- Quando um integrando é caro, reutilizar avaliações entre n e 2n (nós aninhados de Kronrod).
- Medir antes de otimizar: perfil primeiro, WASM só onde o ganho justificar a complexidade.

15.2 Experiência durante a interação

- Primeiro mostrar um resultado grosseiro (malha baixa) e refinar em seguida: o usuário percebe fluidez.
- Mostrar indicador de progresso e botão "cancelar" em tarefas acima de 300 ms.
- Nunca bloquear o clique/arrasto: toda tarefa pesada é assíncrona.
- Manter a última cena válida visível enquanto o novo resultado é calculado ou se a fórmula tiver erro.

16. Publicação e distribuição

- **Build estático** (`pnpm build`) hospedado em Vercel, Netlify ou GitHub Pages, com deploy automático a cada merge na branch principal e previews por Pull Request.
- **Compartilhamento por link:** o estado (JSON) é comprimido (CompressionStream ou lz-string) e codificado em base64url no `location.hash`. Como o hash não é enviado ao servidor, não há dados do usuário na rede. Se o link ficar longo demais, oferecer download do arquivo .json.
- **Segurança:** Content-Security-Policy restritiva (sem scripts de terceiros), validação de esquema do JSON ao abrir, lista branca no gerador de código (Seção 9).
- **PWA:** service worker (vite-plugin-pwa) para uso offline em sala de aula sem internet.
- **SharedArrayBuffer (Fase 8):** exige cabeçalhos `Cross-Origin-Opener-Policy` e `Cross-Origin-Embedder-Policy`; confirme que a hospedagem escolhida permite configurá-los.
- **Desktop opcional:** empacotar com Tauri (WebView do sistema, binário pequeno) se quiser app instalável.
- **Versionamento:** SemVer, CHANGELOG automático a partir dos Conventional Commits, tags por fase.
- **Licença:** MIT é uma escolha simples. Confira as licenças das dependências (Three.js, KaTeX e MathLive são MIT; math.js é Apache-2.0) e mantenha um arquivo THIRD-PARTY-NOTICES.
- **Documentação pública:** README com GIFs, guia de sintaxe das fórmulas, lista de exemplos e FAQ matemático.

17. Riscos e mitigações

Risco	Prob.	Impacto	Mitigação
Escopo cresce sem controle ("só mais uma funcionalidade")	Alta	Alto	MoSCoW, fases com entregável, congelar escopo da fase atual, anotar ideias no backlog em vez de implementar.
Erros numéricos silenciosos (resultado bonito e errado)	Média	Alto	Estimativa de erro (n vs 2n), casos de teste com valor exato, validação cruzada com SymPy, avisos na UI.
Desempenho insuficiente em integrais triplas e malhas grandes	Média	Alto	Workers, resolução adaptativa, orçamentos medidos, WASM nos pontos quentes.
Falta de tempo ou motivação	Média	Alto	Entregas semanais pequenas, demo pública desde a Fase 1, GIFs de progresso, cronograma com folga.
Erros conceituais de matemática	Média	Alto	Revisar com livro-texto/professor; conferir convenções (ADR); gabaritos independentes.
Parsing ambíguo (xy, 2x, sen/sin, expoentes)	Média	Médio	Gramática documentada, pré-visualização LaTeX do que foi entendido, testes extensos.
Artefatos de transparência e ordenação 3D	Alta	Médio	Opacidade moderada, wireframe, OIT na Fase 8, modo "contorno" alternativo.
Conflito de convenções de ângulos esféricos	Média	Médio	ADR + diagrama na interface + opção de alternar convenção.
Segurança do gerador de código (new Function)	Baixa	Alto	Lista branca estrita, fuzzing, CSP, nunca usar eval em texto bruto.
Compatibilidade WebGL/GPU/navegadores/móveis	Média	Médio	Testar Chrome, Firefox, Safari e Android; limites de tamanho de textura/instâncias; fallback gracioso.
Complexidade da interface (muitas opções)	Média	Médio	Modos por tarefa, padrões sensatos, testes com 3 a 5 usuários reais por fase.

18. Backlog de ideias futuras

- **Cálculo vetorial:** integrais de linha (trabalho) e de superfície (fluxo); visualização de Green, Stokes e Gauss.
- **Mudança de variáveis geral:** mapear uma região do plano uv para xy e mostrar o Jacobiano como deformação de uma grade.
- **Séries:** aproximação de Taylor em duas variáveis; séries de Fourier 2D.
- **EDPs:** calor, onda, Laplace, com diferenças finitas e Crank–Nicolson.
- **Sistemas dinâmicos:** diagramas de bifurcação, seções de Poincaré, expoentes de Lyapunov, mapas logísticos.
- **Educação:** modo exercício com gabarito automático, geração de exercícios aleatórios, modo professor com atividades.
- **Colaboração:** sala compartilhada em tempo real (Yjs/WebRTC).
- **Realidade aumentada/virtual:** WebXR para "entrar" no sólido.
- **Biblioteca npm:** publicar `core/` como pacote reutilizável (@integraviz/core).
- **Notebook:** widget para Jupyter usando o mesmo núcleo.
- **WebGPU:** compute shaders para Monte Carlo e milhares de trajetórias de EDO em paralelo.
- **Exportação didática:** gerar PDF/LaTeX com a resolução passo a passo e as figuras da cena.

19. Checklist da primeira semana (dia a dia)

Dia	Meta	Resultado verificável
1	Instalar ferramentas; criar repositório; scaffolding Vite + React + TS; ESLint/Prettier; CI.	Commit inicial com CI verde; pnpm dev abre a página.
2	Hello Three.js: cena, câmera com Z para cima, OrbitControls, eixos simples, superfície fixa.	Superfície z = sen(x)cos(y) girando com o mouse.
3	math.js: parse, gerador de código com lista branca, 20 testes unitários.	pnpm vitest passa; erros em português para casos inválidos.
4	Ligar o campo de texto à superfície; tratar NaN/Infinity; manter última cena válida em caso de erro.	Digitar x^2 - y^2 muda a superfície; digitar "sin(" mostra erro sem quebrar.
5	Web Worker com Comlink, debounce e cancelamento por id.	Malha 256 × 256 sem travar a interface ao digitar.
6	Store Zustand; salvar/abrir JSON; deploy de preview.	Link público funcionando; arquivo reabre a mesma cena.
7	Revisão: ADRs, README, limpeza; planejar a Semana 2 (sliders, cores, picking).	Docs atualizados; issues da semana 2 criadas.

Como estudar em paralelo
Reserve 2 a 3 horas por semana para teoria. Fase 1: noções de GLSL e iluminação. Fase 2: quadratura de Gauss e erro de integração. Fase 4: Hessiana e otimização. Fase 5: Jacobianos e coordenadas. Fase 6: métodos de Runge–Kutta e estabilidade. Implementar um algoritmo logo depois de estudá-lo fixa o conteúdo e deixa o projeto mais correto.

20. Referências e fontes de estudo

Livros

- STEWART, J. *Cálculo*, vol. 2 — integrais múltiplas, cálculo vetorial (capítulos 14 a 16, conforme a edição).
- THOMAS, G. B. *Cálculo*, vol. 2.
- MARSDEN, J.; TROMBA, A. *Vector Calculus*.
- BOYCE, W.; DIPRIMA, R. *Equações Diferenciais Elementares e Problemas de Valores de Contorno*.
- STROGATZ, S. *Nonlinear Dynamics and Chaos* — retratos de fase, bifurcações, Lorenz.
- HAIRER, E.; NØRSETT, S.; WANNER, G. *Solving Ordinary Differential Equations I* — referência para Runge–Kutta.
- BURDEN, R.; FAIRES, J. *Análise Numérica* — quadratura, métodos para EDO.
- PRESS, W. et al. *Numerical Recipes* — Gauss–Legendre, Monte Carlo, raízes.
- TREFETHEN, L. N. "Is Gauss Quadrature Better than Clenshaw–Curtis?", *SIAM Review*, 2008.

Artigos clássicos

- DORMAND, J. R.; PRINCE, P. J. "A family of embedded Runge–Kutta formulae", *J. Comput. Appl. Math.*, 1980.
- GENZ, A. C.; MALIK, A. A. "An adaptive algorithm for numerical integration over an N-dimensional rectangular region", *J. Comput. Appl. Math.*, 1980.

Documentação e tutoriais

- Three.js: documentação e exemplos oficiais (threejs.org), Discover three.js (discoverthreejs.com), Three.js Journey.
- The Book of Shaders (thebookofshaders.com) e os artigos de Inigo Quilez (iquilezles.org) para shaders e SDFs.
- Paul Bourke: página clássica sobre Marching Cubes.
- math.js (mathjs.org): parse, derivative, expressões; MathLive (mathlive.io); KaTeX (katex.org).
- MDN: Web Workers, Transferables, OffscreenCanvas, WebAssembly, CompressionStream.
- Vitest, fast-check e Playwright: documentações oficiais.

Ferramentas para se inspirar (e se diferenciar)

- GeoGebra 3D Calculator e Desmos 3D: referência de usabilidade para entrada de fórmulas e sliders.
- CalcPlot3D: foco em Cálculo III (gradiente, integrais múltiplas).
- Math3D e Wolfram Demonstrations: exemplos de visualização.
- pplane/dfield (Polking): retratos de fase e campos de direções clássicos.

Apêndice A — Folha rápida de fórmulas e convenções

Sistema	Transformação	Elemento de área/volume	Observação
Polar (2D)	$x = r\cos\theta$; $y = r\operatorname{sen}\theta$	$dA = r\,dr\,d\theta$	$\theta \in [0, 2\pi]$; $r \geq 0$
Cilíndricas (3D)	$x = r\cos\theta$; $y = r\operatorname{sen}\theta$; $z = z$	$dV = r\,dz\,dr\,d\theta$	Mesmo θ e r da polar
Esféricas (física/ISO)	$x = \rho\operatorname{sen}\varphi\cos\theta$; $y = \rho\operatorname{sen}\varphi\operatorname{sen}\theta$; $z = \rho\cos\varphi$	$dV = \rho^2\operatorname{sen}\varphi\,d\rho\,d\varphi\,d\theta$	$\varphi \in [0, \pi]$ a partir de $+z$
Esféricas (matemática pura)	troca os papéis de θ e φ	$dV = \rho^2\operatorname{sen}\theta\,d\rho\,d\theta\,d\varphi$	Mostrar a convenção na UI
Geral 2D	$x = x(u, v)$; $y = y(u, v)$	$dA = \lvert\partial(x, y)/\partial(u, v)\rvert\,du\,dv$	Determinante da Jacobiana
Geral 3D	x, y, z em função de u, v, w	$dV = \lvert\partial(x, y, z)/\partial(u, v, w)\rvert\,du\,dv\,dw$	—

Classificação de equilíbrios de sistemas lineares 2D (J = matriz Jacobiana)

Condição ($T = \operatorname{tr} J$; $D = \det J$)	Tipo	Estabilidade
$D < 0$	Sela	Instável
$D > 0$ e $T^2 - 4D \geq 0$	Nó (próprio/impróprio)	Estável se $T < 0$; instável se $T > 0$
$D > 0$ e $T^2 - 4D < 0$ e $T \neq 0$	Foco (espiral)	Estável se $T < 0$; instável se $T > 0$
$D > 0$ e $T = 0$	Centro (linear)	Estável (neutro); pode mudar no sistema não linear
$D = 0$	Degenerado (autovalor nulo)	Requer análise adicional

Critério da segunda derivada (2 variáveis)

Em um ponto crítico: $D = f_{xx}f_{yy} - (f_{xy})^2$. Se $D > 0$ e $f_{xx} > 0$: mínimo local; $D > 0$ e $f_{xx} < 0$: máximo local; $D < 0$: ponto de sela; $D = 0$: inconclusivo.

Apêndice B — Biblioteca de exemplos para o app

B.1 Superfícies e integrais

Nome	Definição	Sugestão de uso
Paraboloide	$z = x^2 + y^2$	Gradiente radial; volume sob a superfície em disco (polar).
Sela	$z = x^2 - y^2$	Ponto crítico de sela; curvas de nível hiperbólicas.
Onda radial	$z = \text{sen}(\sqrt{(x^2 + y^2)})$	Ondas concêntricas; polar.
Sinc (chapéu mexicano)	$z = \text{sen}(r)/r$, $r = \sqrt{(x^2 + y^2)}$ (definir sinc(0) = 1)	Ponto removível; tratar r = 0 como caso especial.
Gaussiana	$z = e^{-(x^2+y^2)}$	Integral sobre o plano todo = π (imprópria; usar discos crescentes).
Caixa de ovos	$z = \text{sen}(x)\cdot\text{sen}(y)$	Muitos máximos e mínimos; classificação de pontos críticos.
Cume e vale	$z = x\cdot e^{-x^2-y^2}$	Um máximo e um mínimo; gradiente.
Cone	$z = \sqrt{(x^2 + y^2)}$	Volume em cilíndricas (T3).
Esfera (paramétrica)	(sen u cos v, sen u sen v, cos u), $u \in [0, \pi]$, $v \in [0, 2\pi]$	Esféricas, ρ = 1.
Toro (paramétrico)	((R + r cos v) cos u, (R + r cos v) sen u, r sen v)	Volume $2\pi^2Rr^2$ (T6).
Faixa de Möbius	$x = (1 + (v/2)\cos(u/2))\cos u$; $y = (1 + (v/2)\cos(u/2))\,\text{sen}\,u$; $z = (v/2)\,\text{sen}(u/2)$; $u \in [0, 2\pi]$; $v \in [-1, 1]$	Superfície não orientável.
Sólido de Steinmetz	$x^2 + y^2 \le 1$ e $x^2 + z^2 \le 1$	Volume 16/3 (T7); implícito.

B.2 Equações diferenciais

Nome	Sistema	Parâmetros típicos
Crescimento exponencial	$y' = k\,y$	k = 1; solução $y = y_0\,e^{kt}$
Logístico	$y' = r\,y\,(1 - y/K)$	r = 1, K = 1
Oscilador harmônico	$x' = v$; $v' = -\omega^2\,x$	ω = 1
Oscilador amortecido	$x' = v$; $v' = -\omega^2\,x - c\,v$	ω = 1; c = 0,2 a 3 (sub, crítico, super)
Pêndulo	$\theta' = \omega$; $\omega' = -(g/L)\,\text{sen}\,\theta - b\,\omega$	g/L = 1; b = 0 a 0,5
Van der Pol	$x' = y$; $y' = \mu(1 - x^2)y - x$	μ = 0,5 a 4 (ciclo limite)
Lotka–Volterra	$x' = a\,x - b\,x\,y$; $y' = d\,x\,y - c\,y$	a = 1, b = 0,1, c = 1,5, d = 0,075
Lorenz	$x' = \sigma(y - x)$; $y' = x(\rho - z) - y$; $z' = x\,y - \beta\,z$	σ = 10, ρ = 28, β = 8/3
Rössler	$x' = -y - z$; $y' = x + a\,y$; $z' = b + z(x - c)$	a = 0,2; b = 0,2; c = 5,7

Nome	Sistema	Parâmetros típicos
Campo de direções simples	$y' = t - y$	Solução exata: $y = t - 1 + C\,e^{-t}$

Apêndice C — Glossário

Termo	Significado
AST	Árvore de sintaxe abstrata: representação em árvore de uma fórmula depois do parse.
Quadratura	Aproximação numérica de uma integral por soma ponderada de valores do integrando.
Gauss–Legendre	Regra de quadratura com nós e pesos ótimos; exata para polinômios de grau até 2n − 1.
Quase-Monte Carlo (QMC)	Integração com sequências de baixa discrepância (Halton, Sobol); converge mais rápido que Monte Carlo puro.
Jacobiano	Determinante da matriz de derivadas de uma mudança de variáveis; fator que corrige área/volume.
Hessiana	Matriz das segundas derivadas parciais; define curvatura local e classifica pontos críticos.
Nuclina / isóclina	Curva onde uma componente do campo é nula / onde a inclinação do campo é constante.
PVI	Problema de valor inicial: EDO + condição inicial.
Marching cubes	Algoritmo que extrai uma malha triangular de uma superfície implícita F = 0 numa grade 3D.
InstancedMesh	Objeto do Three.js que desenha milhares de cópias de uma geometria em uma só chamada de desenho.
Web Worker	Thread separada do navegador para cálculo pesado sem travar a interface.
WASM	WebAssembly: formato binário executado no navegador com desempenho próximo ao nativo.
OIT	Transparência independente de ordem (order-independent transparency).
Debounce	Esperar um intervalo sem novas entradas antes de executar uma tarefa (evita recalcular a cada tecla).
ADR	Architecture Decision Record: nota curta registrando uma decisão técnica e seu motivo.